A New Perspective on Key Switching for BGV-like Schemes

Johannes Mono ●●
johannes.mono@rub.de
0000-0002-0839-058X

Tim Güneysu ●●
tim.gueneysu@rub.de
0000-0002-3293-4989

● Ruhr University Bochum, Bochum, Germany
● CryptoLab Inc., Seoul, Republic of Korea
● DFKI GmbH, Bremen, Germany

Fully homomorphic encryption is a promising approach when computing on encrypted data, especially when sensitive data is involved. For BFV, BGV, and CKKS, three state-of-the-art encryption schemes, the most costly homomorphic primitive is the so-called key switching. While a decent amount of research has been devoted to optimizing other aspects of these schemes, key switching has gone largely untouched. One exception has been a recent work [26] introducing a new double-decomposition technique. Their contributions are an important addition to the current state-of-the-art with one flaw: They take a limited perspective on key switching parameters and their asymptotic complexity which leads to incorrect conclusions about how effective their approach really is. In our work, we deep dive into key switching and correct, enhance, and improve the current state-of-the-art. We provide a new perspective on key switching parameters for the single- and double-decomposition techniques, respectively, and show that the former outperforms the latter in most scenarios. Additionally, we revisit an idea by Gentry, Halevi, and Smart [17] and reduce the number of multiplications.

Cryptography originally had one goal in mind: encrypting messages and ensuring the confidentiality of the encrypted data. Since then, it passed many generations and branched out to a variety of other applicable areas. One such area was first envisioned in the late 1970s under the name privacy homomorphism, a hopeful possibility of arbitrary computations on encrypted data [28]. It is a rather simple idea: A user encrypts (sensitive) data and sends the ciphertext to a powerful server; the server manipulates the ciphertext to compute on the encrypted data; the user decrypts the ciphertext recovering the result. As simple as the idea sounds, realizing it proved to be a tough challenge for many years. Some constructions supported multiplications on encrypted numbers, but no additions. Others supported unlimited additions, but only one multiplication; others many additions, but only a few multiplications. For arbitrary computations, however, any amount of additions and multiplications was needed.

In 2009, Gentry introduced an ingenious idea, bootstrapping, and gave birth to the first ever encryption scheme for privacy homomorphism [15]. Over the years, many more and more efficient schemes have been conceived. Today, they are known as fully homomorphic encryption (FHE) schemes and, at their heart, are still rooted in Gentry's idea of bootstrapping. Conceptually, a ciphertext in any modern FHE scheme has an associated error which grows for each addition or multiplication. Once this error reaches a certain threshold, no further operations are possible without destroying the encrypted numbers. Gentry noticed that we can essentially refresh the associated error given an encryption of the secret key, the so-called bootstrapping. By interleaving operations and bootstrappings appropriately, a server can perform any amount of additions and multiplications on the underlying numbers.

Today's schemes fall into two groups: Boolean-based schemes encrypt single bits or small bit groups, and word-based schemes encrypt large vectors of numbers. While the former enjoys relatively fast bootstrapping and high computational flexibility, the latter suffers from much slower bootstrapping and less flexibility. Word-based schemes however outshine their Boolean-based companions for highly parallelizable arithmetic. Consider vector arithmetic: In word-based schemes, homomorphic addition and multiplication map to the component-wise addition and multiplication of the encrypted vectors of numbers. In Boolean-based schemes, each bit of a vector element would require its own ciphertext and a vast number of homomorphic operations for a vector addition or multiplication. Word-based schemes also support a third primitive, somewhat increasing their computational flexibility: rotations of the encrypted vector. Using rotations, we can map unencrypted

algorithms to the homomorphic realm even if vector elements at different positions need to interact with each other. An example is homomorphic matrix multiplication which requires only two ciphertexts for word-based schemes, one for each matrix operand [23]. In Boolean-based schemes, we would need a seperate ciphertext for every single bit of every matrix element and many homomorphic operations.

Our work focuses on the word-based schemes BFV [6, 14], BGV [7], and CKKS [11]. Due to their similarities, they are known as BGV-like schemes and base their security on the Learning with Errors over Rings (RLWE) assumption. For a ciphertext modulus q and a power-of-two degree N, the ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$ serves as the mathematical foundation of a ciphertext polynomial $c(s)$ with coefficients $c_i \in \mathcal{R}_q$. For decryption, we evaluate a ciphertext polynomial in the secret key s and recover the message m with an additional error e scaled by a known factor t:

$$c(s) = c_0 + c_1 \cdot s = m + te.$$

Obviously, we only publish the coefficients of a ciphertext polynomial and keep the secret key our secret. Homomorphic addition and multiplication straightforwardly map to polynomial addition and multiplication of ciphertexts. Addition adds the messages as $c(s) + c'(s) = m + m' + te_{\text{add}}$ and multiplication multiplies them as $c(s) \cdot c'(s) = mm' + te_{\text{mul}}$. But, as straightforward as it may seem, two issues arise.

Especially for a multiplication, the error grows rather fast. To accomodate it, we require a large ciphertext modulus q sized several hundred bits and, as a consequence, require a large degree N to stay secure in the RLWE setting. The large parameters result in expensive computations and subpar performance. To speed up computations, implementations employ two strategies: First, they decompose the ciphertext modulus into ℓ co-prime q_i with $q = \prod q_i$ enabling computations on the smaller moduli q_i; this is known as residue number system (RNS). Second, they use the forward and inverse number theoretic transform (NTT) for multiplication in $\mathcal{R}_q$. Although these two strategies somewhat ease the computational burden, they do not remove it.

The second issue pertains the output ciphertext and its decryption. The sum

$$(c + c')(s) = (c_0 + c'_0) + (c_1 + c'_1) \cdot s$$

only requires s for decryption. The product

$$(c \cdot c')(s) = (c_0 \cdot c'_0) + (c_0 \cdot c'_1 + c_1 \cdot c'_0) \cdot s + (c_1 \cdot c'_1) \cdot s^2$$

suddenly requires s^2 for decryption and further multiplications would worsen our problem exponentially. BGV-like schemes avoid this ciphertext expansion with an internal housekeeping operation: key switching. Key switching

transforms

$$(c_1 \cdot c_1') \cdot s^2 \quad \mapsto \quad \tilde{c}_0 + \tilde{c}_1 \cdot s + t\tilde{e},$$

holding the same information at the cost of an additional small error $\tilde{e}$. The modified homomorphic multiplication outputs

$$(c \cdot c')(s) = (c_0 \cdot c_0' + \tilde{c}_0) + (c_0 \cdot c_1' + c_1 \cdot c_0' + \tilde{c}_1) \cdot s + t\tilde{e}$$

and only requires s for decryption. We also switch keys after our third primitive operation, rotations. To rotate an encrypted vector, we apply a permutation π on the ciphertext as

$$\pi(c(s)) = \pi(c_0) + \pi(c_1) \cdot \pi(s).$$

As during multiplication, we transform $\pi(c_1) \cdot \pi(s)$ to $\tilde{c}_0 + \tilde{c}_1 \cdot s$ at the cost of an added error. We control this error with two additional parameters: the key switching modulus P and the decomposition number ω. Although there exists a relatively large body of work exploring efficient parameter selection for the security level λ, polynomial degree N, and the ciphertext modulus q, the same cannot be said for the key switching parameters P and ω [2, 12, 1, 13, 27].

1.1 Related Work

Key switching is the most expensive primitive in BGV-like scheme. It occupies roughly $40\,\%$ of execution time during bootstrapping and is $11\times$ slower compared to a ciphertext multiplication without key switching[1]. But, despite its high costs, works on key switching are a rare sight. In the appendix of their extended version, Kim, Polyakov, and Zucca [25] explore the current state-of-the-art on key switching. They describe two different techniques, the BV technique [8] and the GHS technique [16] as well as their combination to the hybrid technique (which we will refer to as single-decomposition technique). They analyse computational and memory complexity, but do not extend their analysis from correct parameter selection to optimal parameter selection. Han and Ki [20] shortly discuss trade-offs for the parameter P on a high-level, but do not show how to choose parameters optimally. Kim et al. [26] propose an extension to the current state-of-the-art with a double-decomposition technique, but with one major shortcoming: a flawed comparison with the single-decomposition technique. Since the initial release of this work, Cheon et al. [9] have built upon our results using mostly large primes and fully decouple scaling in CKKS from the ciphertext modulus with Grafting and so-called universal sprouts.

1.2 Contributions

In this work, we take an in-depth look at key switching and answer the following questions on the state-of-the-art:

1 using our implementation and benchmarking setup, see also Section 4

1. Currently, the double-decomposition technique is known to outperform the single-decomposition technique in relevant parameter ranges. Does it really?
2. Should I always use a given N and q or adjust them for better performance?
3. How do I set the parameters P and ω for maximum performance in practice?

To answer these questions, we make the following contributions:

- We provide a new perspective on the single-decomposition technique with the bound $\mathcal{O}(\omega\ell N \log N)$. We confirm our theoretical results with benchmarks and introduce new guidelines for parameter selection.
- We extend the original work [26] on the double-decomposition technique with the bound $\mathcal{O}((\omega\ell/\tilde{\omega} + \tilde{\omega}\ell/\omega)N \log N)$ and correct the comparison with the single-decomposition technique.
- We integrate an idea by Gentry, Halevi, and Smart [17] and, in our example, speed up key switching by $1.45\times$ on average while only introducing a negligible overhead.
- We apply novel multiplication foldings and speed up key switching by up to $1.16\times$.

Preliminaries
Section 2

Understanding our contributions requires an understanding of key switching and related concepts. For experts, we provide a short summary at the end including our notation (see Subsection 2.6). For more unfamiliar readers, we will revisit key switching and related concepts below.

2.1 RLWE Encryption

In the beginning, there simply is a vector of numbers (our message) that we want to encrypt: integers modulo p for BFV/BGV and approximate numbers for CKKS. Using different paths for the different schemes, our message ends up in a plaintext polynomial $m \in \mathcal{R}_q$; for each coefficient in the most significant bits for BFV and in the least significant bits for BGV and CKKS. But, while plaintext encoding is interesting in itself [11, 19], we simply start with the ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$ and a plaintext $m \in \mathcal{R}_q$. Our encryption toolbox contains a secret key distribution χ_s, an error distribution χ_e, the error scaling factor t (different for each scheme[2]), and the uniform random distri-

2 For BFV, we set $t = 1$ and keep the error in the least significant bits. For BGV, we set $t = p$ and move the error above the message bits. For CKKS, we set $t = 1$ and consider the error as part of the approximation.

bution over $\mathcal{R}_q$. We sample a secret key $s \leftarrow \chi_s$, a small error $e \leftarrow \chi_e$, and a random polynomial $a \leftarrow \mathcal{R}_q$ from our toolbox and encrypt m:

$$(c_0, c_1) = (a \cdot s + te + m, -a) \in \mathcal{R}_q^2.$$

To decrypt, we evaluate the ciphertext $c = (c_0, c_1)$ as polynomial in s:

$$c(s) = c_0 + c_1 \cdot s = m + te \in \mathcal{R}_q.$$

The relevant related works [6, 14, 7, 25, 11, 10, 24] analyze correctness and security.

A BGV-like public key is an encryption of nothing:

$$\mathsf{pk} = (\mathsf{pk}_0, \mathsf{pk}_1) = (a \cdot s + te, -a) \in \mathcal{R}_q^2.$$

For public key encryption, we sample a temporary secret $u \leftarrow \chi_s$, two small errors $e_0, e_1 \leftarrow \chi_e$, and a random polynomial $a \leftarrow \mathcal{R}_q$ from our toolbox and encrypt as

$$(c_0, c_1) = (\mathsf{pk}_0 \cdot u + te_0 + m, \mathsf{pk}_1 \cdot u + te_1) \in \mathcal{R}_q^2.$$

As with a secret key encryption, $c(s) = m + te_{\mathsf{pk}} \in \mathcal{R}_q$ with a slightly larger error e_{pk}.

A BGV-like key switching key is an encryption of a secret s':

$$\mathsf{ksk} = (\mathsf{ksk}_0, \mathsf{ksk}_1) = (a \cdot s + te + s', -a) \in \mathcal{R}_q^2.$$

Key switching aims to output $(\tilde{c}_0, \tilde{c}_1)$ such that

$$\tilde{c}_0 + \tilde{c}_1 \cdot s + t\tilde{e} = c_i \cdot s'$$

adding a small key switching error $\tilde{e}$. To transform $c_i s^2$ after a multiplication, we set $s' = s^2$; to transform $c_i \pi(s)$ after a rotation π, we set $s' = \pi(s)$. In the following, we will explore how to switch keys from s' to s.

2.2 Key Switching

The idea of key switching is rather straightforward: $(\tilde{c}_0, \tilde{c}_1) = (c_i \cdot \mathsf{ksk}_0, c_i \cdot \mathsf{ksk}_1)$. Then,

$$\begin{aligned}\tilde{c}(s) &= c_i \cdot \mathsf{ksk}_0 + c_i \cdot \mathsf{ksk}_1 \cdot s \\ &= c_i \cdot (a \cdot s + te + s') - c_i \cdot a \cdot s \\ &= c_i \cdot s' + tc_i \cdot e\end{aligned}$$

But, it does not quite work: While e is small, the added error $\tilde{e} = c_i \cdot e$ is not because c_i behaves like a random element in $\mathcal{R}_q$. Current state-of-the art

employs two strategies to control the error: modulus extension and decomposition [25]. On our two detours, we will highlight the changes to naïve key switching.

For modulus extension, we temporarily compute in $\mathcal{R}_{qP}$ with the extension modulus P. We modify our naïve key switching key to

$$(\mathsf{ksk}_0, \mathsf{ksk}_1) = (a \cdot s + te + Ps', -a) \in \mathcal{R}_{qP}^2$$

where we sample $a \leftarrow \mathcal{R}_{qP}$. We compute key switching as before, but over $\mathcal{R}_{qP}$, thus:

$$c_i \cdot \mathsf{ksk}_0 + c_i \cdot \mathsf{ksk}_1 \cdot s = c_i \cdot Ps' + tc_i \cdot e \in \mathcal{R}_{qP}.$$

Finally, we switch back to $\mathcal{R}_q$ by scaling by $1/P$ resulting in

$$c_i \cdot s' + t\frac{c_i \cdot e}{P} \in \mathcal{R}_q.$$

For a more straightforward notation, we will only use $\mathcal{R}_q$ and denote other ring moduli explicitely using the notation $[\cdot]_{qP}$. Hence, we switch keys with modulus extension computing

$$(\tilde{c}_0, \tilde{c}_1) = \left(\frac{[c_i \cdot \mathsf{ksk}_0]_{qP}}{P}, \frac{[c_i \cdot \mathsf{ksk}_1]_{qP}}{P}\right).$$

For $P \approx q$, the error $c_i \cdot e/P$ is negligibly small and key switching is correct [25].

For decomposition, we decompose c_i with respect to some base; for example using a power-of-two β with

$$c_i = \sum_{l=0}^{\omega-1} \beta^l c_i^{(l)};$$

the base is $(1, \beta, \beta^2, \ldots)$ and the decomposition $\mathcal{D}(c_i) = (c_i^{(0)}, c_i^{(1)}, c_i^{(2)}, \ldots)$. Next, we modify our naïve key switching key

$$(\mathsf{ksk}_0, \mathsf{ksk}_1) = (a_l \cdot s + te_l + \mathcal{B}(s')_l, -a_l)_{l=0}^{\omega-1} \in \left(\mathcal{R}_q^2\right)^{\omega}$$

with $a_l \leftarrow \mathcal{R}_q$, $e_l \leftarrow \chi_e$, and $\mathcal{B}(s') = (s', \beta s', \beta^2 s', \ldots)$. We also modify key switching itself:

$$(\tilde{c}_0, \tilde{c}_1) = \left(\langle \mathcal{D}(c_i), \mathsf{ksk}_0 \rangle, \langle \mathcal{D}(c_i), \mathsf{ksk}_1 \rangle\right).$$

Then,

$$\begin{aligned}
\tilde{c}(s) &= \langle \mathcal{D}(c_i), \mathsf{ksk}_0 \rangle + \langle \mathcal{D}(c_i), \mathsf{ksk}_1 \rangle \cdot s \\
&= \langle \mathcal{D}(c_i), \mathcal{B}(s') \rangle + t\langle \mathcal{D}(c_i), (e_l)_{l=0}^{\omega-1} \rangle \\
&= c_i \cdot s' + t\sum_{l=0}^{\omega-1} c_i^{(l)} \cdot e_l;
\end{aligned}$$

here, $c_i^{(l)}$ behaves like a random element in $\mathcal{R}_\beta$ instead of $\mathcal{R}_q$ and e_l is small. For small β, key switching is correct [25].

Combining both strategies looks like this:

$$(\mathsf{ksk}_0, \mathsf{ksk}_1) = \left([a_l \cdot s + te_l + P\mathcal{B}(s')_l]_{qP}, [-a_l]_{qP}\right)_{l=0}^{\omega-1} \in \left(\mathcal{R}_{qP}^2\right)^\omega$$

and

$$(\tilde{c}_0, \tilde{c}_1) = \left(\frac{\left[\langle \mathcal{D}(c_i), \mathsf{ksk}_0 \rangle\right]_{qP}}{P}, \frac{\left[\langle \mathcal{D}(c_i), \mathsf{ksk}_1 \rangle\right]_{qP}}{P} \right).$$

Now, the key switching error is negligible as long as P is roughly as large as the largest element in $\mathcal{D}(c_i)$ [25]. Interestingly, for the decomposition, we only need $\mathcal{D}(c_i)$ to be small(ish) and $\langle \mathcal{D}(c_i), \mathcal{B}(s') \rangle = c_i \cdot s'$. Fortunately, implementations benefit from another decomposition that is naturally available.

2.3 DCRT Representation

For BGV-like implementations, we commonly spot two decompositions via the Chinese Remainder Theorem (CRT): the residue number system (RNS) and the number theoretic transform (NTT). The former enables native integer arithmetic modulo large q, and the latter speeds up polynomial multiplication. Combining the two CRT representations is also known as Double CRT (DCRT) representation.

Let $q = \prod_{i=1}^{\ell} q_i$ with co-prime primes q_i. The RNS representation of a polynomial $a \in \mathcal{R}_q$ is $a_i = [a]_{q_i} \in \mathcal{R}_{q_i}$. We can perform additions and multiplications over $\mathcal{R}_q$ in each ring $\mathcal{R}_{q_i}$ individually. Division and modular reduction, however, do not generally map to the RNS space. A notable exception is a scalar $\gamma \in \mathbb{Z}_q$ which divides each coefficient in a, then $a/\gamma = \gamma^{-1}a \in \mathcal{R}_q$. We can reconstruct $a \in \mathcal{R}_q$ from the a_i using the CRT:

$$a = \left(\sum_{i=0}^{\ell} \left[a_i \frac{q_i}{q} \right]_{q_i} \frac{q}{q_i} \right) \in \mathcal{R}_q.$$

The forward and inverse NTT are variants of the Fast Fourier Transform over a finite field and run in time $\mathcal{O}(N \log N)$. The forward NTT transforms a polynomial to the NTT domain, the inverse NTT transforms it back to the coefficient domain. Multiplication in $\mathcal{R}_q$ in the NTT domain corresponds to coefficient-wise multiplication, hence running in time $\mathcal{O}(N)$. Because the NTT is linear, we can move addition and scalar multiplication around independent of a polynomial's domain (within the basic laws of mathematics, of course). In combination with the RNS, we perform the forward and inverse NTT for each ring $\mathcal{R}_{q_i}$ individually. The CRT view on the NTT is as polynomial factorization over $\mathcal{R}_{q_i}$[3]. Since the DCRT representation has a significant impact on key switching, we have to step back and revisit it again.

3 For $q_i = 1 \bmod 2N$, the polynomial $X^N + 1$ splits into N factors $X - \xi^j$ where ξ^j are the odd powers of the $2N$-th root of unity. The NTT is the CRT with respect to these N factors.

2.4 Key Switching, Again

Recall our key switching key

$$(\mathsf{ksk}_0, \mathsf{ksk}_1) = \left([a_l \cdot s + te_l + P\mathcal{B}(s')_l]_{qP}, [-a_l]_{qP}\right)_{l=0}^{\omega-1} \in \left(\mathcal{R}_{qP}^2\right)^{\omega}$$

and key switching itself:

$$(\tilde{c}_0, \tilde{c}_1) = \left(\frac{\left[\langle \mathcal{D}(c_i),\, \mathsf{ksk}_0 \rangle\right]_{qP}}{P}, \frac{\left[\langle \mathcal{D}(c_i),\, \mathsf{ksk}_1 \rangle\right]_{qP}}{P} \right).$$

For the key switching key, we use the RNS with

$$q = \prod_{i=1}^{\ell} q_i \quad \text{and} \quad P = \prod_{j=1}^{k} P_j$$

computing over each $\mathcal{R}_{q_i}$ and $\mathcal{R}_{P_j}$, respectively, and store the result in the NTT domain (we can also sample a_k in the NTT domain). However, instead of decomposing s' to a power-of-two basis β, we use our RNS decomposition over q: We split the ℓ primes q_i into ω groups $\{\tilde{q}_1, \ldots, \tilde{q}_\omega\}$ with up to $\lceil \ell/\omega \rceil$ primes per group. To keep it simple, we will assume $\omega \mid \ell$ for the remainder of this work; hence, each group has ℓ/ω primes. Conceptually, we decompose as

$$\mathcal{D}(c_i) = \left(\left[c_i \frac{\tilde{q}_1}{q} \right]_{\tilde{q}_1}, \ldots, \left[c_i \frac{\tilde{q}_\omega}{q} \right]_{\tilde{q}_\omega} \right) \text{ with } \tilde{q}_l = \prod_{i=1}^{\ell/\omega} q_{(l-1)\ell/\omega + i}.$$

We reconstruct with the CRT, hence

$$\mathcal{B}(s') = \left(\left[s' \frac{q}{\tilde{q}_1} \right]_q, \ldots, \left[s' \frac{q}{\tilde{q}_\omega} \right]_q \right)$$

such that $\langle \mathcal{D}(c_i), \mathcal{B}(s') \rangle = c_i \cdot s'$. The key switching error is negligibly small with $P \approx \tilde{q}_l$ (or simply $k = \ell/\omega$) and key switching is correct, again [25]. Remarkably, we can move the factors $[\tilde{q}_l/q]_{\tilde{q}_l}$ from $\mathcal{D}$ to the key switching key. Thus, $\mathcal{D}$ conceptually transforms the RNS over $\mathcal{R}_q$ to the RNS of each $\mathcal{R}_{\tilde{q}_l}$, respectively, but simply reuses the values in $\mathcal{R}_{q_i}$ [18].

For $\mathcal{D}$, the transition from $\mathcal{R}_q$ to $\mathcal{R}_{\tilde{q}_l}$ is straightforward because $\tilde{q}_l \mid q$. However, converting from one RNS basis to another is not always that easy: For $[\langle \mathcal{D}(c_i), \mathsf{ksk}_0 \rangle]_{qP}$, we need to convert each element in $\mathcal{D}(c_i)$ from $\mathcal{R}_{\tilde{q}_l}$ to $\mathcal{R}_{qP}$, but $qP \nmid \tilde{q}_l$. In general, for two RNS bases

$$E = \prod_{i=1}^{n} E_i \quad \text{and} \quad E' = \prod_{j=1}^{n'} E_j$$

and $a \in \mathcal{R}_E$, the fast base extension

$$\mathsf{BaseExt}(a, E, E') = \left(\left[\sum_{i=0}^{n} \left[a_i \frac{E_i}{E} \right]_{E_i} \frac{E}{E_i} \right]_{E'_j} \right)_{j=1}^{n'}$$

outputs $a_j = [a+\varepsilon E]_{E'_j}$ for a small ε using only native integer arithmetic modulo E_i and E'_j. Often enough, we can consider εE as part of the homomorphic error. If needed, we remove it using an error correction technique such as BEHZ [4] or HPS [18]. For a fast base extension, we need the input in the coefficient domain because we interact between different RNS primes. For key switching, we base extend each element in the decomposition:

$$c_{\text{ext}} = (c_{\text{ext},l})_{l=1}^{\omega} \text{ with } c_{\text{ext},l} = \mathsf{BaseExt}(\mathcal{D}(c_i)_l, \tilde{q}_l, qP).$$

Here, we can consider $\varepsilon\tilde{q}_l$ as part of the negligibly small key switching error [25]. But, this scaling is a division which generally does not map to the RNS space.

Let c'_i be a polynomial in $\mathcal{R}_{qP}$. Our goal is to compute $\tilde{c}_i = [c'_i/P]_q$. While we could simply multiply by $P^{-1} \in \mathbb{Z}_q$ if P would divide every coefficient in c'_i, this is not true in general. But, we can make it true: We add $\delta_i = -t[t^{-1}c'_i]_P$ with $[c'_i+\delta_i]_P = 0$. Because $t \mid \delta_i$ over $\mathcal{R}_q$, it only affects the error. It will also be small because it behaves like a random element in $\mathcal{R}_P$ (roughly P-sized), which we then divide by $1/P$ afterward [25]. Now, P divides every coefficient in $c'_i + \delta_i$ and

$$\tilde{c}_i = \frac{c'_i + \delta_i}{P} = P^{-1}(c'_i + \delta_i).$$

We now move on to a final decomposition.

2.5 Decomposing, Again

Until now, we only used one decomposition $\mathcal{D}$. Recently, Kim et al. [26] suggested a new algorithmic approach using a second decomposition on top. We use the terms single- and double-decomposition technique to differentiate between the two. Their idea is rather straightforward and only uses concepts we already went past.

Recall again single-decomposition key switching:

$$(\tilde{c}_0, \tilde{c}_1) = \left(\frac{\left[\langle c_{\text{ext}}, \mathsf{ksk}_0 \rangle\right]_{qP}}{P}, \frac{\left[\langle c_{\text{ext}}, \mathsf{ksk}_1 \rangle\right]_{qP}}{P} \right).$$

The double-decomposition technique performs the dot product over $\mathcal{R}_{qP}$ in another ring $\mathcal{R}_E$ for a new RNS base E. Initially, we split the $\ell + k$ primes in qP into $\tilde{\omega}$ groups (we will assume $\tilde{\omega} \mid \ell + k$). We decompose ksk toward these $\tilde{\omega}$ groups, afterward extending from each group to $\mathcal{R}_E$ (here, we need to correct the error when using the fast base extension). During key switching, we extend each element in $\mathcal{D}(c_i)$ from $\mathcal{R}_{\tilde{q}_l}$ to $\mathcal{R}_E$ (error correction required) and compute the dot product in $\mathcal{R}_E$[4]. Then, we extend the result to $\mathcal{R}_{qP}$ (no error correction required) and finish up as before by scaling by $1/P$. In the following, we will provide algorithmic descriptions of both techniques after summarizing our notation collected along the way.

4 We could perform the dot product in $\mathcal{R}$ but using the DCRT in $\mathcal{R}_E$ is faster [26]. We choose E large enough to store the dot product without "overflow modulo E".

2.6 Summary

Common notation:

$\mathcal{R}_m$, $[\cdot]_m$	$\mathcal{R}_m = \mathbb{Z}_m[X]/(X^N+1)$ for a power-of-two degree N where $[\cdot]_m$ denotes arithmetic in $\mathcal{R}_m$ (often implicit for $\mathcal{R}_q$)
q, ℓ, b	$q = \prod_{i=1}^{\ell} q_i$ as ciphertext modulus with ℓ co-prime primes q_i with b bits each
P, k, β	$P = \prod_{j=1}^{k} P_j$ as extension modulus with $k = \ell/\omega$ co-prime primes with β bits each; also co-prime to q
E, r, $\tilde{\beta}$	$E = \prod_{i=1}^{r} E_i$ as double-decomposition modulus with r co-prime primes E_i with $\tilde{\beta}$ bits each; also co-prime to q and P
$\mathcal{D}$, $\mathcal{B}$, ω	RNS decomposition $\mathcal{D}(\cdot)$ over $\mathcal{R}_q$ into ω groups $\tilde{q}_l$ such that $\langle \mathcal{D}(c_i), \mathcal{B}(s') \rangle = c_i \cdot s'$
$\tilde{\mathcal{D}}$, $\tilde{\mathcal{B}}$, $\tilde{\omega}$	RNS decomposition $\tilde{\mathcal{D}}(\cdot)$ over $\mathcal{R}_{qP}$ into $\tilde{\omega}$ groups $\tilde{Q}_l$ such that $[\langle \tilde{\mathcal{D}}(c_i), \tilde{\mathcal{B}}(s') \rangle]_{qP} = [c_i \cdot s']_{qP}$
B	upper bound on b, β, and $\tilde{\beta}$
t	the error scaling factor

Common algorithms:

$\mathsf{NTT}_{\mathsf{fwd}}$, $\mathsf{NTT}_{\mathsf{inv}}$	the forward and inverse NTT, respectively
$\mathsf{BaseExt}$	the fast base extension of $a \in \mathcal{R}_E$ to $\mathcal{R}_{E'}$ $$\mathsf{BaseExt}(a, E, E') = \left[\sum_i \left[a_i \frac{E_i}{E} \right]_{E_i} \frac{E}{E_i} \right]_{E'_j}$$

Single-decomposition key switching[5]:

key generation	$\mathsf{ksk} = \left([a_l \cdot s + te_l + P\mathcal{B}(s')_l]_{qP}, [-a_l]_{qP} \right)_{l=0}^{\omega-1}$
input extension	$c_{\mathsf{ext}} = \left(\mathsf{BaseExt}(\mathcal{D}(c_i)_l, \tilde{q}_l, qP) \right)_{l=0}^{\omega-1}$
dot product	$c'_i = \left[\langle \mathsf{NTT}_{\mathsf{fwd}}(c_{\mathsf{ext}}), \mathsf{ksk}_i \rangle \right]_{qP}$

5 We assume that the key switching key ksk is in the NTT domain and do not include the calls to $\mathsf{NTT}_{\mathsf{fwd}}$ and $\mathsf{NTT}_{\mathsf{inv}}$ for key generation. We assume an input c_i in the coefficient domain and output $\tilde{c}_i$ in the coefficient domain. We make the same assumptions for the double-decomposition technique.

scaling	$\delta_i = -t\,\mathsf{BaseExt}(\mathsf{NTT}_{\mathsf{inv}}([t^{-1}c'_i]_P), P, q)$
	$\tilde{c}_i = P^{-1}(\mathsf{NTT}_{\mathsf{inv}}(c'_i) + \delta_i)$

Double-decomposition key switching[6]:

key generation	$\widetilde{\mathsf{ksk}} = \left(\mathsf{BaseExt}(\tilde{\mathcal{D}}(\mathsf{ksk})_l, \tilde{Q}_l, E)\right)_{l=0}^{\tilde{\omega}-1}$
input extension	$c_{\mathsf{ext}} = \left(\mathsf{BaseExt}(\mathcal{D}(c_i)_l, \tilde{q}_l, E)\right)_{l=0}^{\omega-1}$
dot product	$c''_i = \left[\left\langle\left\langle \tilde{\mathcal{B}}(\mathsf{NTT}_{\mathsf{fwd}}(c_{\mathsf{ext}})), \widetilde{\mathsf{ksk}_i}\right\rangle\right\rangle\right]_E$
	$c'_i = \mathsf{BaseExt}\left([\mathsf{NTT}_{\mathsf{inv}}(c''_i)]_E, E, qP\right)$
scaling	$\delta_i = -t\,\mathsf{BaseExt}([t^{-1}c'_i]_P, P, q)$
	$\tilde{c}_i = P^{-1}(\mathsf{NTT}_{\mathsf{inv}}(c'_i) + \delta_i)$

In Appendix B, we map essential notation by Kim et al. [26] to our notation.

Key Switching in Theory
Section 3

Our current understanding of key switching complexity mostly stems from two works [25, 26]. In their extended version, Kim, Polyakov, and Zucca [25] analyse the single-decomposition technique and count the number of forward and inverse NTTs `ntt` (each with complexity $\mathcal{O}(N \log N)$) as well as the number of multiplications `mul`, either coefficient-wise with another polynomial or with a scalar (each with complexity $\mathcal{O}(N)$). They also determine the number of primes `ksk` in a key switching key. In general, they differentiate between BFV and BGV for two reasons:

- The default domain for a ciphertext is different for BFV, BGV, and CKKS. For BFV, a ciphertext is usually in the coefficient domain. For BGV and CKKS, a ciphertext is usually in the NTT domain. Across domains, `ntt` is still the same due to a neat trick that Kim, Polyakov, and Zucca use [25].
- For BFV and CKKS, $t = 1$ reduces the number of scalar multiplications.

For the double-decomposition technique, Kim et al. [26] also provide `ntt`, `mul`, and `ksk`. Additionally, they provide asymptotic complexities for both techniques. We summarize the current state-of-the-art in Table 1. But, there are several problems:

6 With slight abuse of notation, $\langle\langle\cdot\rangle\rangle$ performs a dot product instead of a multiplication for each pair in the outer dot product.

Table 1: Current state-of-the-art on key switching complexity.

	Scheme	Decomposition	
		single	double
$\mathcal{O}$	*	$\mathcal{O}(\ell^2)$	$\mathcal{O}(r^2)$
`ntt`	BFV BGV/CKKS	$(\omega+2)(\ell+k)$	$(\omega+2\tilde{\omega})r$
`mul`	BFV BGV/CKKS	$\ell(\ell+2\omega+2k+5)+2k$ $\ell(\ell+2\omega+2k+7)+4k$	$r(3\ell+2\omega\tilde{\omega}+2k)$
`ksk`	*	$2\omega N(\ell+k)$	$2\omega\tilde{\omega}Nr$

1 For the single-decomposition technique, the perspective on $\mathcal{O}$ is not optimal.

2 There is no estimate for the number of primes r in E that only depends on ℓ, ω, and $\tilde{\omega}$; this complicates comparing both techniques.

3 We can reduce the number of multiplications `mul` in both techniques. Also, Kim et al. [26] exclude the scaling step from `mul`.

4 Kim et al. [26] always assume input and output in the coefficient domain which is only sensible for BFV.

In the following, we will tackle and resolve each issue. Along the way, we will collect questions to evaluate and answer in Section 4.

3.1 A New Perspective

In BGV-like schemes, security depends on the distributions χ_s and χ_e, the degree N, and the modulus qP. For the distributions, common choices are a uniform ternary distribution for χ_s and a centered Gaussian distribution with variance $\sigma = 3.19$ for χ_e [1]. The parameters N, q, and P differ for each use case. Increasing N increases security, but decreases performance. Increasing q (and hence qP) decreases security, but it increases the space where the error can grow and thus permits more homomorphic operations before bootstrapping becomes necessary. In fact, we often try to avoid bootstrapping in BGV-like schemes because it is so expensive, and we instead often choose a large enough q for a use case [17]. Then, we fix a power-of-two N as small as possible for performance, but as large as needed for security[7] [27]. Finally, we choose P and ω such that key switching is correct and secure.

What are the consequences for the key switching parameters P and ω? Essentially, we need to answer the following question: Given N and q, how

7 This almost works: N impacts error growth and hence q, but not significantly.

do we set these parameters for best performance? Kim et al. [26] argue as follows for the single-decomposition key technique (recall that $k = \ell/\omega$): By choosing $k \in \mathcal{O}(1)$, we require $\omega \in \mathcal{O}(\ell)$ and

$$(\omega + 2)(\ell + k) \in \mathcal{O}\big(\ell^2\big)$$

follows accordingly for `ntt`. But, this implicitly limits k. We take a new perspective: We consider $\omega \leq \ell$ as parameter in the security level which we can choose as we desire. Then,

$$(\omega + 2)(\ell + k) = \omega\ell + 3\ell + \frac{2\ell}{\omega} \quad \Rightarrow \quad \texttt{ntt} \in \mathcal{O}(\omega\ell).$$

For $\omega_2 = 2, \omega_1 = 1$,

$$\omega_2\ell + 3\ell + \frac{2\ell}{\omega_2} = \omega_1\ell + 3\ell + \frac{2\ell}{\omega_1},$$

and for $\omega_2 > \omega_1 > 1$,

$$\omega_2\ell + 3\ell + \frac{2\ell}{\omega_2} > \omega_1\ell + 3\ell + \frac{2\ell}{\omega_1}.$$

A simple fact follows: The smaller ω, the better our performance. We also collect our first question to evaluate: In practice, is it better to use $\omega = 1$ or $\omega = 2$?

While the above fact is simple, reality is not, of course. Decreasing ω increases $k = \ell/\omega$ (and hence P (and hence qP)), thus decreasing security for fixed N. Fortunately, there are two solutions: For fixed N, we choose a secure lower bound $\mathcal{W}$ for ω, then set $k \in \mathcal{O}(\ell/\mathcal{W})$ large. Alternatively, we consider the true key switching complexity $\mathcal{O}(\omega\ell N \log N)$ and increase N. For the degree $2N$, we then use $\omega' = 1$ or $\omega' = 2$; we choose $\omega' = 2$. In terms of memory, that is `ksk`, it is worth it to increase the degree once

$$2\omega N(\ell + k) = 2N(\omega\ell + \ell) > 12N\ell = 4N(\omega'\ell + \ell) \quad \Rightarrow \quad \omega > 5.$$

Considering `ntt`, the main computational bottleneck, it gets more complex; we cannot simply use the asymptotic complexity $N \log N$ due to the hidden factors. To keep it simple, we still do: With $2N\log(2N) \approx 2N\log N$, we get

$$\left(\omega\ell + 3\ell + \frac{2\ell}{\omega}\right) N\log N > 12\ell N\log N.$$

We need to solve $\omega^2 - 9\omega + 2 > 0$ and $\omega > 8.77$ follows. At some point, and most likely not at $\omega > 8.77$, increasing the degree should become worth it not only in terms of memory, but also in terms of running time. Does it? A second question to evaluate in Section 4.

3.2 Estimating r

Kim et al. [26] provide a lower bound for $\log_2 E$ based on the infinity norm. An element $a \in \mathcal{R}_m$ has coefficients in $[-m/2, m/2)$, thus $\|a\|_\infty \leq m/2$. For a product $[a \cdot b]_m$, we have

$$\|ab\|_\infty \leq \delta_{\mathcal{R}} \|a\|_\infty \|b\|_\infty$$

for the ring expansion factor $\delta_{\mathcal{R}} = N$ [17]. E needs to be large enough to store a dot product of $\tilde{\omega}$ elements in $\mathcal{R}_{\tilde{q}_l}$ and $\mathcal{R}_{\tilde{Q}_l}$. Thus, the bound is

$$\log_2 E \geq \log_2 \left(\frac{\tilde{\omega} N}{4} \max \tilde{q}_l \max \tilde{Q}_l \right).$$

By definition, we have $\max_l \tilde{q}_l = \ell b / \omega$ and $\max_l \tilde{Q}_l = (\ell b + k\beta)/\tilde{\omega}$. Assuming $b \approx \beta \approx \tilde{\beta} \approx B$, we get

$$r \geq \frac{\log_2(\tilde{\omega} N/4)}{B} + \frac{\ell}{\omega} + \frac{\ell + k}{\tilde{\omega}}.$$

In practice, $\log_2(\tilde{\omega} N/4)/B \in \mathcal{O}(1)$ is negligibly small. With $k = \ell/\omega$, a good estimate is

$$r = \frac{\omega\ell + \tilde{\omega}\ell + \ell}{\omega\tilde{\omega}}.$$

For `ntt`, we get

$$(\omega + 2\tilde{\omega}) r = \left(\frac{\omega^2 + 3\omega\tilde{\omega} + 2\tilde{\omega}^2 + \omega + 2\tilde{\omega}}{\omega\tilde{\omega}} \right) \ell$$

and a key switching complexity $\mathcal{O}((\omega\ell/\tilde{\omega} + \tilde{\omega}\ell/\omega) N \log N)$.

As before, we want to minimize `ntt`, the main computational bottleneck. But, over $\mathbb{R}$, we get negative solutions for ω and $\tilde{\omega}$. We instead exhaustively search for optimal solutions for $\ell \leq 200$, a generous bound for the number of primes in q. For $\ell \leq 200$, we minimize `ntt` with

$$\omega = \ell \quad \text{and} \quad \tilde{\omega} = \sqrt{\left(\frac{\ell + 1}{2} \right) \ell}.$$

But choosing ω and $\tilde{\omega}$ as above is a double-edged sword: It blows up the number of primes in the key switching key

$$2\omega\tilde{\omega} N r = 2(\omega\ell + \tilde{\omega}\ell + \ell) N.$$

In contrast to the single-decomposition technique, we have to use significantly more memory to achieve optimal computational complexity. An important question arises which we answer in Section 4: For optimal parameters, which technique performs better?

3.3 Multiplication Folding

The number of multiplications `mul` consists of two types: coefficient-wise multiplication of two polynomials and scalar multiplication with each coefficient of one polynomial. We need the former only for the dot product with the key switching key. The latter is either a multiplication with a known scheme constant (such as t or P^{-1}) or takes place during the fast base extension

$$\mathsf{BaseExt}(a, E, E') = \left[\sum_i \left[a_i \frac{E_i}{E}\right]_{E_i} \frac{E}{E_i}\right]_{E'_j},$$

one in the source ring $\mathcal{R}_E$ and one in the destination ring $\mathcal{R}_{E'}$ (we precompute the constants). Hence, a multiplication belongs in one of four groups: polynomial multiplication with the key switching key (group 1); scalar multiplication with a known scheme constant (group 2); scalar multiplication during BaseExt in the source ring (group 3); or scalar multiplication during BaseExt in the destination ring (group 4).

Given the right circumstances, we can merge multiplications. In fact, we already encountered an example in Section 2: Moving the factors $[\tilde{q}_l/q]_{\tilde{q}_l}$ from $\mathcal{D}$ (group 3) to the key switching key (group 1). Here, the individual q_i for the source rings $\mathcal{R}_{\tilde{q}_l}$ happen to be all part of the destination ring $\mathcal{R}_{qP}$ and, while conceptually different, they both boil down to computations over all $\mathcal{R}_{q_i}$. In general, the following applies:

- We cannot avoid the multiplication with the key switching key (group 1).
- We can freely move a scalar to the key switching key (group 1) as long as it is needed in every key switching which uses this key switching key; especially known scheme constants (group 2).
- We can move multiplications to and from BaseExt (group 3 and 4) if they are moved for all used RNS primes.

It also does not matter that we have to transform the result of the fast base extension from the coefficient to the NTT domain due to the linearity of the NTT; hence, we now temporarily ignore $\mathsf{NTT}_{\mathsf{fwd}}$ and $\mathsf{NTT}_{\mathsf{inv}}$. Recall single-decomposition key switching starting at the dot product over $\mathcal{R}_{qP}$, and we inline the fast base extension BaseExt:

$\mathcal{R}_q$	$\mathcal{R}_P$
$c'_i = \langle c_{\mathsf{ext}}, \mathsf{ksk}_i \rangle$	$c'_i = \langle c_{\mathsf{ext}}, \mathsf{ksk}_i \rangle$
	$\delta_{i,j} = t^{-1} c'_i P_j / P$
$\delta_i = -t \sum_j \delta_{i,j} P/P_j$	
$\tilde{c}_i = P^{-1}(c'_i + \delta_i).$	

Our crucial observation is that we use the result of the dot product c'_i disjointed over $\mathcal{R}_q$ and $\mathcal{R}_P$: We use $[c']_q$ only to compute $[\tilde{c}_i]_q$ and we use $[c'_i]_P$

only to compute $[\delta_{i,j}]_P$. Thus, we can move scalars around differently for $\mathcal{R}_q$ and $\mathcal{R}_P$, respectively:

$$\begin{aligned} \mathcal{R}_q \qquad\qquad & \qquad\qquad \mathcal{R}_P \\ c'_i = \langle c_{\text{ext}}, P^{-1}\,\mathsf{ksk}_i \rangle \qquad & \qquad \delta_{i,j} = \langle c_{\text{ext}}, t^{-1}P_j/P\,\mathsf{ksk}_i \rangle \\ \delta_i = -t\textstyle\sum_j \delta_{i,j}/P_j & \\ \tilde{c}_i = c'_i + \delta_i. & \end{aligned}$$

Overall, our insights reduce `mul` down to

$$\ell\left(\ell + 2\omega + 2\frac{\ell}{\omega} + 3\right)$$

for any t, that is across all schemes. For a given ℓ, we minimize $2\omega + 2\ell/\omega$, and hence `mul`, with $\omega = \sqrt{\ell}$. Our next question for Section 4: How much time do we gain reducing `mul`?

We also apply our folding techniques to the double-decomposition technique:

$$\begin{aligned} \mathcal{R}_q \qquad\qquad & \qquad\qquad \mathcal{R}_P \qquad\qquad & \mathcal{R}_E \\ & & c''_{i,\iota} = \langle c_{\text{ext}}, E_\iota/E\,\mathsf{ksk}_i \rangle \\ c'_i = P^{-1}\textstyle\sum_\iota c''_{i,\iota}E/E_\iota \qquad & \delta_{i,j} = t^{-1}P_j/P\textstyle\sum_\iota c''_{i,\iota}E/E_\iota & \\ \delta_i = -t\textstyle\sum_j \delta_{i,j}/P_j & & \\ \tilde{c}_i = c'_i + \delta_i. & & \end{aligned}$$

Including scaling, we reduce the number of multiplications down to

$$(\ell + 2\omega\tilde{\omega})r + \ell(2k+3) = \ell\left(2\omega + 2\tilde{\omega} + \frac{3\ell}{\omega} + \frac{\ell}{\tilde{\omega}} + \frac{\ell}{\omega\tilde{\omega}} + 5\right).$$

The local minima of ω and $\tilde{\omega}$ for $0 \le \ell \le 200$ are close to $\sqrt{\ell}$.

3.4 Input and Ouput Domains

Our new foldings have another benefit as part of a bigger picture because we relocated the scaling by P^{-1} from the last step:

$$(\tilde{c}_0, \tilde{c}_1) = (c'_0 + \delta_0, c'_1 + \delta_1).$$

Consider a multiplication where we switch the key of c_2 and output

$$(c_0 + \tilde{c}_0, c_1 + \tilde{c}_1) = (c_0 + c'_0 + \delta_0, c_1 + c'_1 + \delta_1).$$

In the single-decomposition technique, c'_i is in the NTT domain and δ_i is in the coefficient domain. For BFV, we transform c'_i to the coefficient domain and,

for BGV and CKKS, transform δ_i to the NTT domain to match the respective input domain. But sometimes, it can be useful to ignore the default domain. For c_i in the coefficient domain, we can choose the output domain:

$$\mathsf{NTT}_{\mathsf{inv}}(c_i') + c_i + \delta_i \quad \text{or} \quad c_i' + \mathsf{NTT}_{\mathsf{fwd}}(c_i + \delta_i).$$

For c_i in the NTT domain, the same idea works:

$$\mathsf{NTT}_{\mathsf{inv}}(c_i' + c_i) + \delta_i \quad \text{or} \quad c_i' + c_i + \mathsf{NTT}_{\mathsf{fwd}}(\delta_i).$$

Trivially, we can also choose output domains for a rotation where we switch c_1 and output $(c_0 + \tilde{c}_0, \tilde{c}_1)$. Fun fact: We could even choose different output domains for each prime q_i individually.

For the double-decomposition technique, we also moved scaling by P^{-1}:

$$(\tilde{c}_0, \tilde{c}_1) = (c_0' + \delta_0, c_1' + \delta_1);$$

but now, c_i' and δ_i are both in the coefficient domain. For output in the NTT domain, we need 2ℓ additional NTTs:

$$\big(\mathsf{NTT}_{\mathsf{fwd}}(c_0' + \delta_0), \mathsf{NTT}_{\mathsf{fwd}}(c_1' + \delta_1)\big),$$

possibly adding c_0 and/or c_1 before performing the forward NTT. To make things worse, the trick that keeps `ntt` the same across input domains does not work for the double-decomposition technique. For BGV and CKKS, we overall need 3ℓ additional NTTs, 2ℓ forward and ℓ inverse.

3.5 Large Primes, Mostly

So far, we (and all related work) use the general assumption $b \approx \beta \approx \tilde{\beta} \approx B$ for the primes in q, P, and E. But is this reasonable? Generally, choosing primes as close to the upper bound B as possible is beneficial for two obvious reasons:

- Each additional prime increases the number of polynomial operations during homomorphic evaluation. The larger each prime, the fewer primes we need, reducing compute and memory costs.
- Using small primes typically wastes compute and memory resources: operations are still performed over B-sized numbers.

Given a bound $\log_2 q$ (or $\log_2 P$ or $\log_2 E$), we ideally want to:

1. Set $\ell = \lceil \log_2 q / B \rceil$.
2. Choose b such that $\log_2 q \approx \ell \cdot b$.
3. Generate ℓ primes close to 2^b.

For P and E, this is exactly what we do! For BFV, this also works because our input to key switching is always in $\mathcal{R}_q$ so the size of each q_i does not really

matter [25]. For BGV and CKKS, however, we only start in $\mathcal{R}_q$ and remove individual primes over time.

We already encountered the idea of removing primes: scaling by $1/P$ during key switching, which is also known as modulus switching. In BGV, we use modulus switching to reduce the absolute error after a multiplication. We scale by one of the primes q_i and remove it from the ciphertext modulus $q/q_i = q'$. Afterward, we continue in the ring $\mathcal{R}_{q'}$. Relative to q', the error has the same size as before. But the absolute size is scaled by $1/q_i$ and, for properly chosen q_i, we stop the error from growing exponentially in the following multiplications [7]. In CKKS, we solve a different problem with rescaling. During a multiplication, the approximate message moves from the least significant bits of the ciphertext modulus q into higher bits. We then scale by $1/q_i$ to move it back to less significant bits, also removing q_i from the ciphertext modulus. Here, the size of the primes q_i is even more important because scaling has to be pretty precise [10]. For both schemes, scaling by roughly 2^B may not be what we need.

We still need to show that $b \approx \beta \approx \tilde{\beta} \approx B$ is a reasonable assumption. Gentry, Halevi, and Smart outline a high-level approach that we adopt: The modulus q is composed of mostly large primes, supplemented by a few carefully selected small primes that are dynamically included or excluded to enable precise control over scaling [17]. Since they are sparse on the details and do not estimate the costs, we will add these details and estimate costs.

Switching the modulus.

We scale and round $a \in \mathcal{R}_q$ to the modulus q' with

$$a' = \left[\left\lceil \frac{q'}{q} a \right\rfloor_t\right]_{q'}$$

where $\lceil \cdot \rfloor_t$ rounds to the nearest integer coefficients which are $[0]_t$; this keeps the message intact for BFV and BGV. The RNS-friendly version is

$$\left[\left\lceil \frac{q'}{q} a \right\rfloor_t\right]_{q'} = \left[\frac{q'a + \delta}{q}\right]_{q'} = \left[\frac{q'a - t[t^{-1}q'a]_q}{q}\right]_{q'}$$

with $[\delta]_t = 0$ and $q \mid (q'a + \delta)$ by definition. This scales either the error (BFV and BGV) or the approximate message (CKKS) by roughly q'/q. We will use modulus switching to remove specific primes q_i from q. To easily exclude primes from q, we denote $q_{x,y} = \prod_{i=x}^{y} q_i$ with $q = q_{1,\ell}$.

Switching primes in and out.

For $\ell = \ell' - \mu + \kappa$ and $\log_2 q \approx (\ell' + \mu)b$, we use $\ell' - \kappa$ primes close to 2^b as well as $\kappa > \mu$ smaller primes, each close to $2^{\mu b/\kappa}$. For $b \approx B$, we continuously scale by $2^{\mu b/\kappa}$ as follows:

1 Receive a fresh encryption from the client.

2 Perform the desired homomorphic operations until we need to scale.
3 Switch the modulus using one of the κ small primes.
4 Repeat steps (2) and (3) until all κ small primes are gone.
5 Perform homomorphic operations; during the last key switching before scaling, replace any μ large primes with the κ small primes.
6 Continue with step (3) using the small primes we switched back in.

Integrating prime switching with key switching.

If we want to replace the primes $\{q_{\ell-\mu}, \dots, q_\ell\}$ with $\{q_1, \dots, q_\kappa\}$, we switch the modulus of $a \in R_{q_{\kappa+1,\ell}}$ as

$$\begin{aligned}\left[\left\lceil \frac{q_{1,\ell-\mu}}{q_{\kappa+1,\ell}} a \right\rfloor_t\right]_{q_{1,\ell-\mu}} &= \left[\frac{q_{1,\ell-\mu}a - t[t^{-1}q_{1,\ell-\mu}a]_{q_{\kappa+1,\ell}}}{q_{\kappa+1,\ell}}\right]_{q_{1,\ell-\mu}} \\ &= \left[\frac{q_{1,\kappa}a - t[t^{-1}q_{1,\kappa}a]_{q_{\ell-\mu,\ell}}}{q_{\ell-\mu,\ell}}\right]_{q_{1,\ell-\mu}}.\end{aligned}$$

We then integrate with key switching during the scaling step by switching in the small primes $\{q_1, \dots, q_\kappa\}$ and switching out the large ciphertext and key switching primes $\{q_{\ell-\mu}, \dots, q_\ell, P_1, \dots, P_k\}$:

$$\left[\left\lceil \frac{q_{1,\ell-\mu}}{q_{\kappa+1,\ell}P_{1,k}} a \right\rfloor_t\right]_{q_{1,\ell-\mu}} = \left[\frac{q_{1,\kappa}c_i' - t[t^{-1}q_{1,\kappa}c_i']_{q_{\ell-\mu,\ell}P_{1,k}}}{q_{\ell-\mu,\ell}P_{1,k}}\right]_{q_{1,\ell-\mu}}.$$

In the single-decomposition technique, the number of inverse NTT increases from $2k$ to $2(\mu + k)$ for base extending $\delta_i = -t[t^{-1}q_{1,\kappa}c_i']_{q_{\ell-\mu,\ell}P_{1,k}}$. However, we also save 2μ NTT operations on either c_i' or δ_i since we reduce the number of primes for the modulus switching output from ℓ to $\ell - \mu$. In the double-decomposition technique, we are in the coefficient domain anyway after base extending from E. Hence, for output in the coefficient domain, switching primes in and out requires no additional NTT operations. For output in the NTT domain, we need to perform 2κ additional forward NTT operations for both techniques, κ per δ_i. As $\kappa \in \mathcal{O}(1)$ is usually small (think $\kappa = 2$ or $\kappa = 3$), this is a very small overhead. But how small is it in practice? Another question we answer in Section 4.

Choosing the primes.

We modify the parameter selection process as follows:

1 Choose $b \approx B$, μ, and κ to scale by $q_{\mathsf{scale}} = 2^{\mu b/\kappa}$.
2 Set $q = q_{\mathsf{scale}}^L + q_{\mathsf{dec}}$ for L available scalings and a decryption cushion q_{dec}. Also set $\ell' = \lceil \log_2 q/B \rceil$.
3 Generate $\ell' - \mu$ primes close to 2^b.

4 Generate κ primes close to q_{scale}.

5 Set $\ell = \ell' - \mu + \kappa$.

Compared to the more naïve approach, we reduce ℓ as long as

$$\left\lceil \frac{\log_2 q}{\mu b/\kappa} \right\rceil \approx \left\lceil \frac{\ell'}{\mu/\kappa} \right\rceil > \ell \;\Leftrightarrow\; \kappa\ell' - \mu\ell \geq \mu \;\Leftrightarrow\; \ell \geq \kappa + \frac{\mu}{\kappa - \mu}.$$

For example, with $B = 60$, we consider a use case scaling by $q_{\text{scale}} \approx 2^{36}$. Then, with $b = 54 \approx B$, $\mu = 2$, and $\kappa = 3$, we reduce the overall number of primes as soon as $\ell \geq 5$ (equivalent to $\log q > 144$). Our last question for Section 4: How large are our gains when choosing (mostly) large primes?

3.6 Summary

Our goal was to address four general challenges:

1 For the single-decomposition technique, the perspective on $\mathcal{O}$ is not optimal.

2 There is no estimate for the number of primes r in E that only depends on ℓ, ω, and $\tilde{\omega}$; this complicates comparing both techniques.

3 We can reduce the number of multiplications `mul` in both techniques.

4 Kim et al. [26] always assume input and output in the coefficient domain, which usually only holds for BFV, and exclude the scaling step for `mul`.

We resolved all four challenges and also showed that $b \approx \beta \approx \tilde{\beta} \approx B$ is a reasonable assumption even when scaling by smaller primes is required. We emphasize that current implementations of BGV and CKKS either choose large primes and waste ciphertext modulus or choose mostly small primes with subpar performance; the only exception we are aware of is a closed-source implementation by Cheon et al. [9] building upon our results. We update the previous state-of-the-art from Table 1 to our new results in Table 2.

Key Switching in Practice
Section 4

In Section 3, we analyzed key switching in-depth and posed six questions. We evaluate them on an Intel Xeon Gold 6242 with 480 GB of memory (single-threaded). Our BGV implementation[8] uses a HEXL backend [22] for fast polynomial arithmetic. In Appendix C, we also provide results for all benchmarks using a proprietary ring backend on Apple's M3 Pro with 18 GB of memory (single-threaded). For $N \in \{2^{14}, 2^{15}, 2^{16}, 2^{17}\}$, we use $\log_2 qP \leq$

8 https://github.com/Chair-for-Security-Engineering/owl

Table 2: Our update for key switching analysis.

	Scheme	Decomposition	
		single	double
$\mathcal{O}$	*	$\mathcal{O}\big(\omega\ell\, N \log N\big)$	$\mathcal{O}\big((\omega\ell/\tilde{\omega} + \tilde{\omega}\ell/\omega) N \log N\big)$
`ntt`	BFV BGV/CKKS	$\omega\ell + 3\ell + 2\ell/\omega$	$\frac{\omega^2+3\omega\tilde{\omega}+2\tilde{\omega}^2+\omega+2\tilde{\omega}}{\omega\tilde{\omega}}\ell$ $\mathtt{ntt}_{\mathsf{BFV}} + 3\ell$
`mul`	BFV BGV/CKKS	$\ell(\ell + 2\omega + 2\ell/\omega + 3)$	$\ell(2\omega + 2\tilde{\omega} + \frac{\omega\ell+3\tilde{\omega}\ell+\ell}{\omega\tilde{\omega}} + 5)$
`ksk`	*	$2(\omega\ell + \ell)N$	$2(\omega\ell + \tilde{\omega}\ell + \ell)N$

Table 3: Execution times for $\omega = 1$ and $\omega = 2$ where q uses approximately half of the available modulus space. See also Table 10.

Parameters		Time (ms)	
$\log_2 N$	$\log_2 q$	$\omega = 1$	$\omega = 2$
14	180 (41.86 %)	1.9	2.5
15	420 (48.39 %)	12.3	15.3
16	840 (48.08 %)	61.2	74.9
17	1740 (49.39 %)	362.6	416.9

$\{430, 868, 1747, 3523\}$ for at least 128 bit security [5]. If not otherwise noted, we evenly split the available modulus space such that $b = \beta = \tilde{\beta} = B = 60$.

1 Is $\omega = 1$ or $\omega = 2$ better if we can choose (single-decomposition)?

For $\omega = 1$, we require $q \approx P$ and thus q needs to use at most half the available modulus space. For each N, we compare $\omega = 1$ and $\omega = 2$; the results are in Table 3. If possible, using $\omega = 1$ outperforms $\omega = 2$.[9] However, for $\omega = 1$, we could most likely reduce the degree as well; this is essentially the inverse of increasing N which we evaluate and answer next.

2 Can increasing N actually be worth it (single-decomposition)?

Increasing the degree N only makes sense for a large ω and we limit benchmarks to $k = 1$ or $k = 2$; each sibling parameter set

9 In a previous version of this work, $\omega = 2$ mostly outperformed $\omega = 1$ with $\omega = 2$ being closer to the optimal multiplication complexity $\sqrt{\ell}$. Due to an improved implementation and a faster benchmarking platform, reading the key switching key from memory is now more expensive than the constant multiplications; hence, the smaller key with $\omega = 1$ outperforms $\omega = 2$.

Table 4: Execution times for parameter sets where $k = 2$ or $k = 1$ with q using the rest of the available modulus space. Each parameter set has a sibling with degree $2N$ and $\omega' = 1$. We also report the speed-up of $2N$ compared to N. See also Table 11.

Parameters				Time (ms)		
$\log_2 N$	$\log_2 q$	ω	ω'	N	$2N$	Speedup
14	300 (69.77 %)	3	1	5.1	7.8	0.65
14	360 (83.72 %)	6	1	8.9	9.8	0.91
15	720 (82.95 %)	6	1	47.5	51.0	0.93
15	780 (89.86 %)	13	1	91.5	54.7	1.67
16	1620 (92.73 %)	14	1	488.2	327.3	1.49
16	1680 (96.16 %)	28	1	869.2	338.6	2.57
17	3360 (95.37 %)	28	1	3941.7	1917.7	2.06
17	3420 (97.08 %)	57	1	7346.3	1979.1	3.71

uses degree $2N$ and $\omega' = 1$. Table 4 shows that increasing the degree can actually increase key switching performance. But, doing so has costs outside of key switching: larger polynomials to store and to compute on. While we consider this insight mostly a curious fact, it could prove useful in rare circumstances.

3 Is the single- or the double-decomposition technique better?

Kim et al. [26] already compare both techniques using a comprehensive set of benchmarks. They find that the double-decomposition technique mostly outperforms the single-decomposition technique. This is true for their parameters using the same ω. But, this is not how we actually choose parameters: Given N and q, we choose the optimal ω for the single-decomposition technique, or we choose the optimal ω and $\tilde{\omega}$ for the double-decomposition technique. Hence, we have to compare the best ω for each technique and not the same ω. In Figure 1, we normalize execution times to compare the single-decomposition and double-decomposition techniques with $k = 1$ for all ℓ (top, the best-case scenario for the double-decomposition technique) and with optimal parameters per ℓ (bot); lower is better. In most scenarios, the single-decomposition technique outperforms the double-decomposition technique, especially considering that we remove primes over time in BGV and CKKS.[10] Assuming $k \geq 2$ for $N \geq 2^{15}$ and

10 For $k = 1$ in BGV and CKKS, we can also use level-aware key switching in lower levels [21].

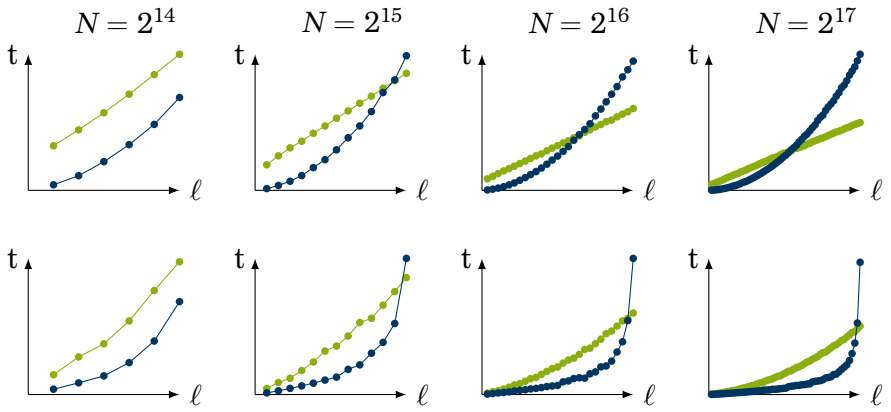

Figure 1: Normalized execution times per number of primes ℓ for the ● single-decomposition and ● double-decomposition techniques (lower is better). We either fix $k = 1$, $\omega = L$, and $\tilde{\omega} = \sqrt{L(L+1)/2}$ maximizing q (top) or set these optimal per ℓ (bot). See also Figure 2. The absolute timings are listed in Table 6.

$k \geq 3$ for $N \geq 2^{17}$, the single-decomposition technique always performs better (see also Figure 2; on the M3 Pro, which has better absolute execution times, the single-decomposition technique is only worse for $N = 2^{17}$ with $k = 1$).

4 How large is the speedup using mostly large primes?

Similar to Kim et al. [26], we use $q_{\text{scale}} \approx 2^{36}$ which matches our example from Subsection 3.5 with $b = 54 \approx B$, $\mu = 2$, and $\kappa = 3$. Our observed speedups range from $1.27\times$ to $1.64\times$ (Table 7 in Appendix A); on average, we speed up key switching by $1.45\times \approx \kappa/\mu\times$. At the top level, we reduce execution times by 4.29 ms, 47.27 ms, 368.52 ms, and 4219.92 ms for degrees 2^{14}, 2^{15}, 2^{16}, and 2^{17}, respectively.

5 How costly is replacing large with small primes?

The median overhead for replacing large primes with small primes is 1.84 ms, 6.58 ms, 10.81 ms, and 60.04 ms for degrees 2^{14}, 2^{15}, 2^{16}, and 2^{17}, respectively (Table 8 in Appendix A). Given the speedups, these overheads are indeed negligible, especially since they only occur once per κ levels.

6 How large are the speedups from constant folding?

In Table 5, we observe speedups up to $1.16\times$. See also Table 9.

4.1 Limitations

We want to mention three limitations of our work:

1 For the single-decomposition technique, choosing ω as small as

Table 5: Speedups of constant foldings for different ω. Table 9 in Appendix A lists the absolute timings. See also Table 15.

$\log_2 N$	ω				
	1	2	3	4	5
14	1.16×	1.13×	1.11×	1.11×	1.08×
15	1.03×	1.10×	1.11×	1.04×	1.06×
16	1.08×	1.08×	1.05×	1.03×	1.01×
17	1.06×	1.02×	1.02×	1.03×	1.01×

possible will result in best performance. For the double-decomposition technique, the trade-off between the number of NTTs and the key size complicates parameter selection. Optimally, we would benchmark the individual operations ($\mathsf{NTT}_{\mathsf{fwd}}$, $\mathsf{NTT}_{\mathsf{inv}}$, coefficient-wise polynomial multiplication, coefficient-wise scalar multiplication, fetching ksk from memory) and optimize parameters specifically to an implementation and platform. But, doing so heavily depends on the platform and we still expect similar results.

2 We use the HPS method [18] for correcting the error after BaseExt if needed; the HPS method tends to perform better than BEHZ [4, 3]. However, benchmarking results using the BEHZ method would still be interesting.

3 An open problem is finding a closed formula for an optimal multiplication complexity in the double-decomposition technique, our best approximation remains roughly $\sqrt{\ell}$.

Conclusion
Section 5

In this work, we deep dive into the current state-of-the-art in key switching and thoroughly analyse its complexity. Along the way, we provided a new perspective on the single- and double-decomposition techniques with the bounds $\mathcal{O}(\omega\ell N \log N)$ and $\mathcal{O}((\omega\ell/\tilde{\omega} + \tilde{\omega}\ell/\omega)N \log N)$. We also provide an estimate for the number of primes r in E (double-decomposition technique), reduce the number of scalar multiplications (both techniques), and correct the number of NTTs for BGV and CKKS (double-decomposition technique). We revisit an idea by Gentry, Halevi, and Smart [17], integrate it with key switching with low overhead, and show that $b \approx \beta \approx \tilde{\beta} \approx B$ can be efficiently combined with small scaling. We confirm our results in practice: Reducing the number of multiplication speeds up key switching by up to 1.16×

and choosing mostly large primes on average by approximately $\kappa/\mu\times$. In the beginning, we also promised answers to three questions:

1 Currently, the double-decomposition technique is known to outperform the single-decomposition technique in relevant parameter ranges. Does it really?
In most scenarios, the single-decomposition technique actually outperforms the double-decomposition technique. It is, however, a cool idea that future work can hopefully improve upon.

2 Should I always use a given N and q or adjust them for better performance?
You most likely do not want to increase N. But you definitely want to adjust q for BGV and CKKS: Use as many large primes as possible! Add some small ones for scaling and replace them with large ones during key switching; it can massively boost performance.

3 How do I set the parameters P and ω for maximum performance in practice?
Given N and q, simply choose ω as small as possible within your security requirements.

Acknowledgements We would like to thank Damien Stehlé for his helpful remarks. The work in this paper has been supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972.

References

[1] Martin R. Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin E. Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. "Homomorphic Encryption Standard". In: IACR Cryptol. ePrint Arch. 2019.939 (2019). URL: https://eprint.iacr.org/2019/939.

[2] Martin R. Albrecht, Rachel Player, and Sam Scott. "On the concrete hardness of Learning with Errors". In: J. Math. Cryptol. 9.3 (2015), pp. 169–203. URL: http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml.

[3] Ahmad Al Badawi, Yuriy Polyakov, Khin Mi Mi Aung, Bharadwaj Veeravalli, and Kurt Rohloff. "Implementation and Performance Evaluation of RNS Variants of the BFV Homomorphic Encryption Scheme". In: IEEE Trans. Emerg. Top. Comput. 9.2 (2021), pp. 941–956. DOI: 10.1109/TETC.2019.2902799. URL: https://doi.org/10.1109/TETC.2019.2902799.

[4] Jean-Claude Bajard, Julien Eynard, M. Anwar Hasan, and Vincent Zucca. “A Full RNS Variant of FV Like Somewhat Homomorphic Encryption Schemes”. In: Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John’s, NL, Canada, August 10-12, 2016, Revised Selected Papers. Ed. by Roberto Avanzi and Howard M. Heys. Vol. 10532. Lecture Notes in Computer Science. Springer, 2016, pp. 423–442. DOI: 10.1007/978-3-319-69453-5_23. URL: https://doi.org/10.1007/978-3-319-69453-5_23.

[5] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and Donggeon Yhee. “Security Guidelines for Implementing Homomorphic Encryption”. In: IACR Commun. Cryptol. 1.4 (2024), p. 26. DOI: 10.62056/ANXRA69P1. URL: https://doi.org/10.62056/anxra69p1.

[6] Zvika Brakerski. “Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP”. In: Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings. Ed. by Reihaneh Safavi-Naini and Ran Canetti. Vol. 7417. Lecture Notes in Computer Science. Springer, 2012, pp. 868–886. DOI: 10.1007/978-3-642-32009-5_50. URL: https://doi.org/10.1007/978-3-642-32009-5_50.

[7] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. “(Leveled) Fully Homomorphic Encryption without Bootstrapping”. In: ACM Trans. Comput. Theory 6.3 (2014), 13:1–13:36. DOI: 10.1145/2633600. URL: https://doi.org/10.1145/2633600.

[8] Zvika Brakerski and Vinod Vaikuntanathan. “Fully Homomorphic Encryption from Ring-LWE and Security for Key Dependent Messages”. In: Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2011. Proceedings. Ed. by Phillip Rogaway. Vol. 6841. Lecture Notes in Computer Science. Springer, 2011, pp. 505–524. DOI: 10.1007/978-3-642-22792-9_29. URL: https://doi.org/10.1007/978-3-642-22792-9%5C_29.

[9] Jung Hee Cheon, Hyeongmin Choe, Minsik Kang, Jaehyung Kim, Seonghak Kim, Johannes Mono, and Taeyeong Noh. *Grafting: Complementing RNS in CKKS*. Cryptology ePrint Archive, Paper 2024/1014. 2024. URL: https://eprint.iacr.org/2024/1014.

[10] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. “A Full RNS Variant of Approximate Homomorphic Encryption”. In: Selected Areas in Cryptography - SAC 2018 - 25th International Conference, Calgary, AB, Canada, August 15-17, 2018, Revised Selected Papers. Ed. by Carlos Cid and Michael J. Jacobson Jr. Vol. 11349. Lecture Notes in Computer Science. Springer, 2018, pp. 347–368. DOI: 10.1007/978-3-030-10970-7_16. URL: https://doi.org/10.1007/978-3-030-10970-7_16.

[11] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. In: Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I. Ed. by Tsuyoshi Takagi and Thomas Peyrin. Vol. 10624. Lecture Notes in Computer Science. Springer, 2017, pp. 409–437.

DOI: 10.1007/978-3-319-70694-8_15. URL: https://doi.org/10.1007/978-3-319-70694-8_15.

[12] Ana Costache and Nigel P. Smart. "Which Ring Based Somewhat Homomorphic Encryption Scheme is Best?" In: Topics in Cryptology - CT-RSA 2016 - The Cryptographers' Track at the RSA Conference 2016, San Francisco, CA, USA, February 29 - March 4, 2016, Proceedings. Ed. by Kazue Sako. Vol. 9610. Lecture Notes in Computer Science. Springer, 2016, pp. 325–340. DOI: 10.1007/978-3-319-29485-8_19. URL: https://doi.org/10.1007/978-3-319-29485-8_19.

[13] Anamaria Costache, Kim Laine, and Rachel Player. "Evaluating the Effectiveness of Heuristic Worst-Case Noise Analysis in FHE". In: Computer Security - ESORICS 2020 - 25th European Symposium on Research in Computer Security, ESORICS 2020, Guildford, UK, September 14-18, 2020, Proceedings, Part II. Ed. by Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider. Vol. 12309. Lecture Notes in Computer Science. Springer, 2020, pp. 546–565. DOI: 10.1007/978-3-030-59013-0_27. URL: https://doi.org/10.1007/978-3-030-59013-0_27.

[14] Junfeng Fan and Frederik Vercauteren. "Somewhat Practical Fully Homomorphic Encryption". In: IACR Cryptol. ePrint Arch. (2012), p. 144. URL: http://eprint.iacr.org/2012/144.

[15] Craig Gentry. "Fully homomorphic encryption using ideal lattices". In: Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009, Bethesda, MD, USA, May 31 - June 2, 2009. Ed. by Michael Mitzenmacher. ACM, 2009, pp. 169–178. DOI: 10.1145/1536414.1536440. URL: https://doi.org/10.1145/1536414.1536440.

[16] Craig Gentry, Shai Halevi, and Nigel P. Smart. "Fully Homomorphic Encryption with Polylog Overhead". In: Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings. Ed. by David Pointcheval and Thomas Johansson. Vol. 7237. Lecture Notes in Computer Science. Springer, 2012, pp. 465–482. DOI: 10.1007/978-3-642-29011-4_28. URL: https://doi.org/10.1007/978-3-642-29011-4_28.

[17] Craig Gentry, Shai Halevi, and Nigel P. Smart. "Homomorphic Evaluation of the AES Circuit". In: Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings. Ed. by Reihaneh Safavi-Naini and Ran Canetti. Vol. 7417. Lecture Notes in Computer Science. Springer, 2012, pp. 850–867. DOI: 10.1007/978-3-642-32009-5_49. URL: https://doi.org/10.1007/978-3-642-32009-5_49.

[18] Shai Halevi, Yuriy Polyakov, and Victor Shoup. "An Improved RNS Variant of the BFV Homomorphic Encryption Scheme". In: Topics in Cryptology - CT-RSA 2019 - The Cryptographers' Track at the RSA Conference 2019, San Francisco, CA, USA, March 4-8, 2019, Proceedings. Ed. by Mitsuru Matsui. Vol. 11405. Lecture Notes in Computer Science. Springer, 2019, pp. 83–105. DOI: 10.1007/978-3-030-12612-4_5. URL: https://doi.org/10.1007/978-3-030-12612-4_5.

[19] Shai Halevi and Victor Shoup. "Design and implementation of HElib: a homomorphic encryption library". In: IACR Cryptol. ePrint Arch. (2020), p. 1481. URL: https://eprint.iacr.org/2020/1481.

[20] Kyoohyung Han and Dohyeong Ki. “Better Bootstrapping for Approximate Homomorphic Encryption”. In: Topics in Cryptology - CT-RSA 2020 - The Cryptographers’ Track at the RSA Conference 2020, San Francisco, CA, USA, February 24-28, 2020, Proceedings. Ed. by Stanislaw Jarecki. Vol. 12006. Lecture Notes in Computer Science. Springer, 2020, pp. 364–390. DOI: 10.1007/978-3-030-40186-3_16. URL: https://doi.org/10.1007/978-3-030-40186-3_16.

[21] Intak Hwang, Jinyeong Seo, and Yongsoo Song. “Optimizing HE operations via Level-aware Key-switching Framework”. In: Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography, Copenhagen, Denmark, 26 November 2023. Ed. by Michael Brenner, Anamaria Costache, and Kurt Rohloff. ACM, 2023, pp. 59–67. DOI: 10.1145/3605759.3625263. URL: https://doi.org/10.1145/3605759.3625263.

[22] Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe DM de Souza, Vinodh Gopal, et al. *Intel HEXL (release 1.2)*. https://github.com/intel/hexl. Sept. 2021.

[23] Xiaoqian Jiang, Miran Kim, Kristin E. Lauter, and Yongsoo Song. “Secure Outsourced Matrix Computation and Application to Neural Networks”. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM, 2018, pp. 1209–1222. DOI: 10.1145/3243734.3243837. URL: https://doi.org/10.1145/3243734.3243837.

[24] Andrey Kim, Antonis Papadimitriou, and Yuriy Polyakov. “Approximate Homomorphic Encryption with Reduced Approximation Error”. In: Topics in Cryptology - CT-RSA 2022 - Cryptographers’ Track at the RSA Conference 2022, Virtual Event, March 1-2, 2022, Proceedings. Ed. by Steven D. Galbraith. Vol. 13161. Lecture Notes in Computer Science. Springer, 2022, pp. 120–144. DOI: 10.1007/978-3-030-95312-6_6. URL: https://doi.org/10.1007/978-3-030-95312-6%5C_6.

[25] Andrey Kim, Yuriy Polyakov, and Vincent Zucca. “Revisiting Homomorphic Encryption Schemes for Finite Fields”. In: Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part III. Ed. by Mehdi Tibouchi and Huaxiong Wang. Vol. 13092. Lecture Notes in Computer Science. Springer, 2021, pp. 608–639. DOI: 10.1007/978-3-030-92078-4_21. URL: https://doi.org/10.1007/978-3-030-92078-4_21.

[26] Miran Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. “Accelerating HE Operations from Key Decomposition Technique”. In: Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part IV. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14084. Lecture Notes in Computer Science. Springer, 2023, pp. 70–92. DOI: 10.1007/978-3-031-38551-3_3. URL: https://doi.org/10.1007/978-3-031-38551-3%5C_3.

[27] Johannes Mono, Chiara Marcolla, Georg Land, Tim Güneysu, and Najwa Aaraj. “Finding and Evaluating Parameters for BGV”. In: Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings. Ed. by Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne. Vol. 14064. Lecture Notes in Computer Science.

Springer, 2023, pp. 370–394. DOI: 10.1007/978-3-031-37679-5_16. URL: https://doi.org/10.1007/978-3-031-37679-5_16.

[28] Ronald Rivest, Len Adleman, and Michael Dertouzos. “On data banks and privacy homomorphisms”. In: Foundations of secure computation 4.11 (1978), pp. 169–180.

Evaluation Listings

Table 6: Absolute execution times comparing the single- and double-decomposition technique in Figure 1.

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	14	6	1	6	0	9.2
single	14	6	1	6	0	9.2
double	14	6	1	6	5	13.4
double	14	6	1	6	5	13.1
single	14	5	1	5	0	6.5
single	14	5	2	3	0	5.3
double	14	5	1	5	5	11.4
double	14	5	1	5	4	10.3
single	14	4	1	4	0	4.5
single	14	4	2	2	0	3.2
double	14	4	1	4	5	9.5
double	14	4	1	4	3	7.3
single	14	3	1	3	0	2.8
single	14	3	3	1	0	1.8
double	14	3	1	3	5	7.7
double	14	3	1	3	2	5.0
single	14	2	1	2	0	1.4
single	14	2	2	1	0	1.2
double	14	2	1	2	5	6.0
double	14	2	1	2	2	3.7
single	14	1	1	1	0	0.5
single	14	1	1	1	0	0.5
double	14	1	1	1	5	4.4
double	14	1	1	1	1	2.0
single	15	13	1	13	0	91.5
single	15	13	1	13	0	92.6
double	15	13	1	13	10	79.6
double	15	13	1	13	10	79.6
single	15	12	1	12	0	75.1
single	15	12	2	6	0	48.3

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	15	12	1	12	10	74.6
double	15	12	1	12	9	70.0
single	15	11	1	11	0	66.5
single	15	11	3	4	0	35.5
double	15	11	1	11	10	69.4
double	15	11	1	11	8	60.8
single	15	10	1	10	0	53.6
single	15	10	4	3	0	28.1
double	15	10	1	10	10	64.0
double	15	10	1	10	7	51.8
single	15	9	1	9	0	43.8
single	15	9	5	2	0	20.2
double	15	9	1	9	10	59.2
double	15	9	1	9	7	49.0
single	15	8	1	8	0	35.8
single	15	8	4	2	0	17.1
double	15	8	1	8	10	54.0
double	15	8	1	8	6	39.8
single	15	7	1	7	0	27.6
single	15	7	7	1	0	11.8
double	15	7	1	7	10	49.0
double	15	7	1	7	5	33.0
single	15	6	1	6	0	20.8
single	15	6	6	1	0	9.8
double	15	6	1	6	10	43.8
double	15	6	1	6	5	29.6
single	15	5	1	5	0	15.4
single	15	5	5	1	0	7.7
double	15	5	1	5	10	38.8
double	15	5	1	5	4	22.1
single	15	4	1	4	0	10.0
single	15	4	4	1	0	5.9
double	15	4	1	4	10	33.4
double	15	4	1	4	3	16.3
single	15	3	1	3	0	6.2
single	15	3	3	1	0	4.2
double	15	3	1	3	10	28.9
double	15	3	1	3	2	11.0
single	15	2	1	2	0	3.3
single	15	2	2	1	0	2.5
double	15	2	1	2	10	23.4
double	15	2	1	2	2	8.2

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	15	1	1	1	0	1.2
single	15	1	1	1	0	1.1
double	15	1	1	1	10	17.4
double	15	1	1	1	1	4.1
single	16	28	1	28	0	868.6
single	16	28	1	28	0	914.8
double	16	28	1	28	20	550.2
double	16	28	1	28	20	547.9
single	16	27	1	27	0	810.6
single	16	27	2	14	0	496.6
double	16	27	1	27	20	528.4
double	16	27	1	27	19	511.8
single	16	26	1	26	0	762.5
single	16	26	3	9	0	354.4
double	16	26	1	26	20	512.0
double	16	26	1	26	19	495.2
single	16	25	1	25	0	697.6
single	16	25	4	7	0	285.4
double	16	25	1	25	20	494.1
double	16	25	1	25	18	460.2
single	16	24	1	24	0	646.1
single	16	24	5	5	0	224.7
double	16	24	1	24	20	471.9
double	16	24	1	24	17	426.7
single	16	23	1	23	0	595.0
single	16	23	6	4	0	192.1
double	16	23	1	23	20	461.3
double	16	23	1	23	17	409.0
single	16	22	1	22	0	546.3
single	16	22	6	4	0	185.6
double	16	22	1	22	20	442.9
double	16	22	1	22	16	371.7
single	16	21	1	21	0	500.5
single	16	21	7	3	0	152.0
double	16	21	1	21	20	425.7
double	16	21	1	21	15	341.7
single	16	20	1	20	0	447.5
single	16	20	7	3	0	139.4
double	16	20	1	20	20	406.2
double	16	20	1	20	14	311.8
single	16	19	1	19	0	409.0
single	16	19	10	2	0	110.0

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	16	19	1	19	20	390.1
double	16	19	1	19	14	302.7
single	16	18	1	18	0	374.5
single	16	18	9	2	0	112.5
double	16	18	1	18	20	365.2
double	16	18	1	18	13	269.9
single	16	17	1	17	0	336.9
single	16	17	9	2	0	107.6
double	16	17	1	17	20	353.3
double	16	17	1	17	12	244.6
single	16	16	1	16	0	293.9
single	16	16	8	2	0	91.2
double	16	16	1	16	20	333.0
double	16	16	1	16	12	232.1
single	16	15	1	15	0	261.8
single	16	15	8	2	0	85.3
double	16	15	1	15	20	319.0
double	16	15	1	15	11	210.9
single	16	14	1	14	0	230.4
single	16	14	14	1	0	61.1
double	16	14	1	14	20	300.8
double	16	14	1	14	10	184.9
single	16	13	1	13	0	203.4
single	16	13	13	1	0	57.4
double	16	13	1	13	20	283.5
double	16	13	1	13	10	173.8
single	16	12	1	12	0	175.8
single	16	12	12	1	0	53.6
double	16	12	1	12	20	265.5
double	16	12	1	12	9	155.6
single	16	11	1	11	0	149.9
single	16	11	11	1	0	46.8
double	16	11	1	11	20	244.8
double	16	11	1	11	8	135.0
single	16	10	1	10	0	126.1
single	16	10	10	1	0	41.5
double	16	10	1	10	20	231.7
double	16	10	1	10	7	115.1
single	16	9	1	9	0	105.9
single	16	9	9	1	0	37.6
double	16	9	1	9	20	214.3
double	16	9	1	9	7	106.4

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	16	8	1	8	0	85.5
single	16	8	8	1	0	32.1
double	16	8	1	8	20	196.3
double	16	8	1	8	6	89.3
single	16	7	1	7	0	67.7
single	16	7	7	1	0	28.1
double	16	7	1	7	20	179.9
double	16	7	1	7	5	73.4
single	16	6	1	6	0	49.8
single	16	6	6	1	0	23.8
double	16	6	1	6	20	162.3
double	16	6	1	6	5	64.3
single	16	5	1	5	0	36.5
single	16	5	5	1	0	18.1
double	16	5	1	5	20	145.9
double	16	5	1	5	4	49.5
single	16	4	1	4	0	24.7
single	16	4	4	1	0	13.7
double	16	4	1	4	20	131.0
double	16	4	1	4	3	36.9
single	16	3	1	3	0	15.1
single	16	3	3	1	0	10.7
double	16	3	1	3	20	112.9
double	16	3	1	3	2	25.7
single	16	2	1	2	0	7.7
single	16	2	2	1	0	6.6
double	16	2	1	2	20	94.2
double	16	2	1	2	2	19.1
single	16	1	1	1	0	2.8
single	16	1	1	1	0	3.0
double	16	1	1	1	20	77.4
double	16	1	1	1	1	9.5
single	17	57	1	57	0	7546.3
single	17	57	1	57	0	7334.0
double	17	57	1	57	41	3760.2
double	17	57	1	57	41	3789.2
single	17	56	1	56	0	7181.7
single	17	56	2	28	0	3962.9
double	17	56	1	56	41	3697.0
double	17	56	1	56	40	3636.2
single	17	55	1	55	0	6846.3
single	17	55	3	19	0	2851.8

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	17	55	1	55	41	3651.7
double	17	55	1	55	39	3484.8
single	17	54	1	54	0	6609.1
single	17	54	4	14	0	2269.9
double	17	54	1	54	41	3592.7
double	17	54	1	54	39	3441.8
single	17	53	1	53	0	6388.4
single	17	53	5	11	0	1889.9
double	17	53	1	53	41	3516.1
double	17	53	1	53	38	3312.3
single	17	52	1	52	0	6079.7
single	17	52	6	9	0	1644.8
double	17	52	1	52	41	3465.3
double	17	52	1	52	37	3187.8
single	17	51	1	51	0	5835.8
single	17	51	7	8	0	1519.1
double	17	51	1	51	41	3403.8
double	17	51	1	51	36	3053.6
single	17	50	1	50	0	5683.5
single	17	50	8	7	0	1388.4
double	17	50	1	50	41	3332.0
double	17	50	1	50	36	2994.0
single	17	49	1	49	0	5414.0
single	17	49	9	6	0	1253.5
double	17	49	1	49	41	3269.4
double	17	49	1	49	35	2872.8
single	17	48	1	48	0	5214.3
single	17	48	10	5	0	1128.8
double	17	48	1	48	41	3235.1
double	17	48	1	48	34	2765.9
single	17	47	1	47	0	4962.2
single	17	47	10	5	0	1109.1
double	17	47	1	47	41	3167.6
double	17	47	1	47	34	2700.8
single	17	46	1	46	0	4790.6
single	17	46	12	4	0	981.1
double	17	46	1	46	41	3106.1
double	17	46	1	46	33	2584.7
single	17	45	1	45	0	4624.7
single	17	45	12	4	0	941.9
double	17	45	1	45	41	3030.9
double	17	45	1	45	32	2464.1

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	17	44	1	44	0	4429.2
single	17	44	11	4	0	917.3
double	17	44	1	44	41	2971.1
double	17	44	1	44	31	2354.1
single	17	43	1	43	0	4169.1
single	17	43	15	3	0	820.2
double	17	43	1	43	41	2905.0
double	17	43	1	43	31	2319.3
single	17	42	1	42	0	4047.8
single	17	42	14	3	0	769.8
double	17	42	1	42	41	2868.3
double	17	42	1	42	30	2228.8
single	17	41	1	41	0	3867.3
single	17	41	14	3	0	748.4
double	17	41	1	41	41	2803.9
double	17	41	1	41	29	2101.6
single	17	40	1	40	0	3624.1
single	17	40	14	3	0	729.6
double	17	40	1	40	41	2749.3
double	17	40	1	40	29	2030.2
single	17	39	1	39	0	3464.9
single	17	39	13	3	0	696.9
double	17	39	1	39	41	2683.6
double	17	39	1	39	28	1952.4
single	17	38	1	38	0	3292.5
single	17	38	19	2	0	589.5
double	17	38	1	38	41	2615.7
double	17	38	1	38	27	1855.9
single	17	37	1	37	0	3119.0
single	17	37	19	2	0	582.4
double	17	37	1	37	41	2557.7
double	17	37	1	37	27	1804.1
single	17	36	1	36	0	3001.3
single	17	36	18	2	0	547.3
double	17	36	1	36	41	2474.4
double	17	36	1	36	26	1706.5
single	17	35	1	35	0	2828.9
single	17	35	18	2	0	542.4
double	17	35	1	35	41	2420.9
double	17	35	1	35	25	1613.0
single	17	34	1	34	0	2684.0
single	17	34	17	2	0	506.0

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	17	34	1	34	41	2368.8
double	17	34	1	34	24	1541.1
single	17	33	1	33	0	2501.7
single	17	33	17	2	0	502.9
double	17	33	1	33	41	2326.7
double	17	33	1	33	24	1488.9
single	17	32	1	32	0	2359.3
single	17	32	16	2	0	472.8
double	17	32	1	32	41	2317.0
double	17	32	1	32	23	1404.4
single	17	31	1	31	0	2241.2
single	17	31	16	2	0	461.3
double	17	31	1	31	41	2187.5
double	17	31	1	31	22	1319.2
single	17	30	1	30	0	2085.0
single	17	30	15	2	0	428.8
double	17	30	1	30	41	2135.4
double	17	30	1	30	22	1282.9
single	17	29	1	29	0	1947.2
single	17	29	29	1	0	352.9
double	17	29	1	29	41	2074.0
double	17	29	1	29	21	1202.7
single	17	28	1	28	0	1831.4
single	17	28	28	1	0	338.8
double	17	28	1	28	41	2050.6
double	17	28	1	28	20	1126.7
single	17	27	1	27	0	1701.7
single	17	27	27	1	0	323.2
double	17	27	1	27	41	1975.1
double	17	27	1	27	19	1038.7
single	17	26	1	26	0	1585.1
single	17	26	26	1	0	306.0
double	17	26	1	26	41	1882.1
double	17	26	1	26	19	1004.7
single	17	25	1	25	0	1471.2
single	17	25	25	1	0	290.0
double	17	25	1	25	41	1829.0
double	17	25	1	25	18	949.7
single	17	24	1	24	0	1362.1
single	17	24	24	1	0	291.3
double	17	24	1	24	41	1798.3
double	17	24	1	24	17	872.5

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	17	23	1	23	0	1257.9
single	17	23	23	1	0	263.5
double	17	23	1	23	41	1721.7
double	17	23	1	23	17	831.5
single	17	22	1	22	0	1158.1
single	17	22	22	1	0	248.6
double	17	22	1	22	41	1644.6
double	17	22	1	22	16	783.9
single	17	21	1	21	0	1062.2
single	17	21	21	1	0	248.8
double	17	21	1	21	41	1593.1
double	17	21	1	21	15	719.9
single	17	20	1	20	0	965.9
single	17	20	20	1	0	219.4
double	17	20	1	20	41	1522.0
double	17	20	1	20	14	659.5
single	17	19	1	19	0	879.9
single	17	19	19	1	0	206.5
double	17	19	1	19	41	1459.6
double	17	19	1	19	14	629.8
single	17	18	1	18	0	794.5
single	17	18	18	1	0	192.8
double	17	18	1	18	41	1385.1
double	17	18	1	18	13	573.4
single	17	17	1	17	0	712.5
single	17	17	17	1	0	175.3
double	17	17	1	17	41	1326.5
double	17	17	1	17	12	506.7
single	17	16	1	16	0	638.0
single	17	16	16	1	0	164.8
double	17	16	1	16	41	1263.8
double	17	16	1	16	12	487.9
single	17	15	1	15	0	564.3
single	17	15	15	1	0	151.6
double	17	15	1	15	41	1206.0
double	17	15	1	15	11	431.9
single	17	14	1	14	0	489.0
single	17	14	14	1	0	139.4
double	17	14	1	14	41	1131.3
double	17	14	1	14	10	384.2
single	17	13	1	13	0	448.6
single	17	13	13	1	0	126.0

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	17	13	1	13	41	1070.5
double	17	13	1	13	10	361.9
single	17	12	1	12	0	376.7
single	17	12	12	1	0	114.5
double	17	12	1	12	41	1023.2
double	17	12	1	12	9	325.0
single	17	11	1	11	0	316.8
single	17	11	11	1	0	102.9
double	17	11	1	11	41	931.1
double	17	11	1	11	8	277.1
single	17	10	1	10	0	264.9
single	17	10	10	1	0	91.6
double	17	10	1	10	41	885.9
double	17	10	1	10	7	236.1
single	17	9	1	9	0	232.9
single	17	9	9	1	0	81.9
double	17	9	1	9	41	824.6
double	17	9	1	9	7	218.3
single	17	8	1	8	0	176.3
single	17	8	8	1	0	70.1
double	17	8	1	8	41	762.2
double	17	8	1	8	6	183.1
single	17	7	1	7	0	140.9
single	17	7	7	1	0	59.4
double	17	7	1	7	41	700.5
double	17	7	1	7	5	162.5
single	17	6	1	6	0	108.0
single	17	6	6	1	0	52.6
double	17	6	1	6	41	632.9
double	17	6	1	6	5	132.9
single	17	5	1	5	0	79.1
single	17	5	5	1	0	39.3
double	17	5	1	5	41	565.1
double	17	5	1	5	4	104.6
single	17	4	1	4	0	53.6
single	17	4	4	1	0	30.5
double	17	4	1	4	41	504.7
double	17	4	1	4	3	78.1
single	17	3	1	3	0	32.4
single	17	3	3	1	0	22.4
double	17	3	1	3	41	447.2
double	17	3	1	3	2	53.3

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	17	2	1	2	0	16.8
single	17	2	2	1	0	14.0
double	17	2	1	2	41	384.5
double	17	2	1	2	2	40.2
single	17	1	1	1	0	6.2
single	17	1	1	1	0	6.5
double	17	1	1	1	41	326.3
double	17	1	1	1	1	19.9

Table 7: Absolute execution times comparing only small and mostly large primes in Section 4.

Parameters				Time (ms)		
$\log_2 N$	ℓ_{36}	ℓ_{54}	ω	`small`	`large`	Speedup
14	9	6	6	13.4	9.1	1.47
14	6	4	2	4.5	3.2	1.39
14	3	2	1	1.6	1.2	1.35
15	21	14	14	151.9	104.7	1.45
15	18	12	4	56.6	37.6	1.51
15	15	10	2	31.1	22.0	1.42
15	12	8	2	23.9	16.7	1.43
15	9	6	1	14.6	9.6	1.53
15	6	4	1	8.3	5.8	1.43
15	3	2	1	3.5	2.5	1.40
16	45	30	30	1393.3	1024.8	1.36
16	42	28	10	593.8	406.1	1.46
16	39	26	6	411.2	282.1	1.46
16	36	24	4	295.1	200.5	1.47
16	33	22	3	238.6	162.0	1.47
16	30	20	2	180.8	119.9	1.51
16	27	18	2	154.2	105.9	1.46
16	24	16	2	134.3	90.7	1.48
16	21	14	1	87.2	61.0	1.43
16	18	12	1	75.2	55.2	1.36
16	15	10	1	57.7	40.8	1.41
16	12	8	1	43.7	32.8	1.33
16	9	6	1	31.1	23.2	1.34
16	6	4	1	19.9	14.5	1.38
16	3	2	1	8.8	6.1	1.43
17	96	64	64	13296.0	9076.0	1.46
17	93	62	21	5338.0	3575.8	1.49

$\log_2 N$	ℓ_{36}	ℓ_{54}	ω	`small`	`large`	Speedup
17	90	60	12	3630.6	2343.4	1.55
17	87	58	9	2992.9	1911.9	1.57
17	84	56	7	2546.1	1590.1	1.60
17	81	54	5	2133.0	1314.5	1.62
17	78	52	4	1809.7	1128.4	1.60
17	75	50	4	1760.0	1089.8	1.61
17	72	48	3	1511.0	923.6	1.64
17	69	46	3	1400.6	882.7	1.59
17	66	44	3	1304.0	830.5	1.57
17	63	42	2	1063.1	692.6	1.53
17	60	40	2	995.8	659.4	1.51
17	57	38	2	945.7	604.3	1.56
17	54	36	2	857.4	549.7	1.56
17	51	34	2	796.0	514.6	1.55
17	48	32	1	584.5	417.1	1.40
17	45	30	1	534.6	369.5	1.45
17	42	28	1	472.5	340.1	1.39
17	39	26	1	427.0	308.6	1.38
17	36	24	1	387.1	275.4	1.41
17	33	22	1	349.9	244.9	1.43
17	30	20	1	304.1	218.2	1.39
17	27	18	1	259.7	192.3	1.35
17	24	16	1	231.5	164.8	1.40
17	21	14	1	188.7	139.9	1.35
17	18	12	1	160.4	119.6	1.34
17	15	10	1	126.2	91.5	1.38
17	12	8	1	96.8	70.3	1.38
17	9	6	1	68.2	49.8	1.37
17	6	4	1	43.6	31.3	1.39
17	3	2	1	20.2	15.9	1.27

Table 8: Absolute execution times and overhead of switching out large for small primes in Section 4.

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`folded`	`switch`	Overhead
14	6	6	9.1	11.7	2.6
14	4	2	3.2	5.1	1.8
14	2	1	1.2	2.8	1.6
15	14	14	104.7	112.5	7.9
15	12	4	37.6	45.4	7.8

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`folded`	`switch`	Overhead
15	10	2	22.0	28.7	6.7
15	8	2	16.7	22.9	6.1
15	6	1	9.6	16.1	6.6
15	4	1	5.8	10.6	4.9
15	2	1	2.5	6.6	4.1
16	30	30	1024.8	1023.3	-1.5
16	28	10	406.1	417.8	11.7
16	26	6	282.1	288.0	5.9
16	24	4	200.5	214.8	14.3
16	22	3	162.0	175.0	13.0
16	20	2	119.9	131.6	11.6
16	18	2	105.9	116.7	10.8
16	16	2	90.7	102.0	11.3
16	14	1	61.0	74.7	13.7
16	12	1	55.2	63.0	7.7
16	10	1	40.8	52.8	11.9
16	8	1	32.8	42.9	10.1
16	6	1	23.2	32.0	8.7
16	4	1	14.5	23.3	8.8
16	2	1	6.1	14.7	8.5
17	64	64	9076.0	9308.0	232.0
17	62	21	3575.8	3665.8	90.0
17	60	12	2343.4	2442.3	98.9
17	58	9	1911.9	1987.9	76.0
17	56	7	1590.1	1683.6	93.5
17	54	5	1314.5	1388.9	74.4
17	52	4	1128.4	1214.2	85.8
17	50	4	1089.8	1159.6	69.8
17	48	3	923.6	1018.4	94.8
17	46	3	882.7	967.6	84.9
17	44	3	830.5	894.7	64.2
17	42	2	692.6	756.1	63.6
17	40	2	659.4	708.4	49.0
17	38	2	604.3	672.6	68.3
17	36	2	549.7	618.7	69.0
17	34	2	514.6	578.1	63.5
17	32	1	417.1	461.5	44.4
17	30	1	369.5	429.5	60.0
17	28	1	340.1	394.7	54.6
17	26	1	308.6	356.8	48.1
17	24	1	275.4	324.5	49.1

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`folded`	`switch`	Overhead
17	22	1	244.9	291.2	46.3
17	20	1	218.2	255.3	37.1
17	18	1	192.3	224.1	31.9
17	16	1	164.8	195.0	30.2
17	14	1	139.9	180.4	40.5
17	12	1	119.6	141.5	22.0
17	10	1	91.5	117.0	25.5
17	8	1	70.3	93.6	23.3
17	6	1	49.8	72.2	22.4
17	4	1	31.3	52.5	21.2
17	2	1	15.9	33.0	17.1

Table 9: Absolute execution times of constant folding speedups in Table 5.

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`naïve`	`folded`	Speedup
14	3	1	2.1	1.8	1.16
14	4	2	3.6	3.1	1.13
14	3	3	3.0	2.7	1.11
14	4	4	4.8	4.3	1.11
14	5	5	7.0	6.5	1.08
14	6	6	9.2	8.9	1.03
15	7	1	12.9	12.4	1.03
15	8	2	18.5	16.7	1.10
15	9	3	25.7	23.1	1.11
15	8	4	24.6	23.6	1.04
15	10	5	37.4	35.5	1.06
15	12	6	49.0	49.1	1.00
15	7	7	28.8	28.1	1.02
15	8	8	37.5	35.9	1.04
15	9	9	45.7	45.0	1.02
15	10	10	57.3	56.6	1.01
16	14	1	64.3	59.6	1.08
16	18	2	108.9	101.0	1.08
16	21	3	151.0	143.1	1.05
16	20	4	157.4	152.8	1.03
16	20	5	181.0	178.6	1.01
16	24	6	245.1	239.3	1.02
16	21	7	231.8	225.6	1.03
16	24	8	290.6	289.4	1.00

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`naïve`	`folded`	Speedup
16	18	9	229.3	221.6	1.03
16	20	10	273.6	268.2	1.02
17	29	1	376.9	356.6	1.06
17	38	2	612.2	600.4	1.02
17	42	3	796.0	781.5	1.02
17	44	4	933.1	905.5	1.03
17	45	5	1050.6	1036.7	1.01
17	48	6	1251.6	1205.1	1.04
17	49	7	1383.3	1352.4	1.02
17	48	8	1444.7	1400.5	1.03
17	45	9	1425.8	1377.2	1.04
17	50	10	1717.3	1664.7	1.03

Mapping Our Notation to Kim et al. [26]

Ours	[26]	Description
N	N	polynomial degree
q	Q	ciphertext modulus
P	P	key switching modulus
qP	$\tilde{Q}$	ciphertext and key switching modulus
E	B	double-decomposition modulus
ℓ	ℓ	number of primes in q
k	r	number of primes in P
r	r'	number of primes in E
$\ell + k$	$\tilde{\ell}$	number of primes in qP
ω	d	number of decompositions over q
$\tilde{\omega}$	$\tilde{d}$	number of decomposition over qP
-	$\tilde{r}$	number of primes in each group in the decomposition over qP

Evaluation on the M3 Pro

Table 12: Absolute M3 Pro execution times comparing the single- and double-decomposition technique in Figure 2.

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	14	6	1	6	0	4.4
single	14	6	1	6	0	4.4

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	14	6	1	6	5	6.7
double	14	6	1	6	5	6.7
single	14	5	1	5	0	3.1
single	14	5	2	3	0	2.8
double	14	5	1	5	5	5.7
double	14	5	1	5	4	5.1
single	14	4	1	4	0	2.2
single	14	4	2	2	0	1.9
double	14	4	1	4	5	4.9
double	14	4	1	4	3	3.9
single	14	3	1	3	0	1.5
single	14	3	3	1	0	1.2
double	14	3	1	3	5	4.0
double	14	3	1	3	2	2.8
single	14	2	1	2	0	1.0
single	14	2	2	1	0	0.9
double	14	2	1	2	5	3.2
double	14	2	1	2	2	2.2
single	14	1	1	1	0	0.7
single	14	1	1	1	0	0.7
double	14	1	1	1	5	2.4
double	14	1	1	1	1	1.3
single	15	13	1	13	0	21.2
single	15	13	1	13	0	21.2
double	15	13	1	13	10	27.2
double	15	13	1	13	10	27.2
single	15	12	1	12	0	18.5
single	15	12	2	6	0	14.3
double	15	12	1	12	10	25.4
double	15	12	1	12	9	23.7
single	15	11	1	11	0	15.8
single	15	11	3	4	0	11.6
double	15	11	1	11	10	23.7
double	15	11	1	11	8	20.5
single	15	10	1	10	0	12.9
single	15	10	4	3	0	9.5
double	15	10	1	10	10	21.8
double	15	10	1	10	7	17.3
single	15	9	1	9	0	10.6
single	15	9	5	2	0	7.5
double	15	9	1	9	10	20.0
double	15	9	1	9	7	16.0

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	15	8	1	8	0	8.9
single	15	8	4	2	0	6.1
double	15	8	1	8	10	18.4
double	15	8	1	8	6	13.3
single	15	7	1	7	0	7.4
single	15	7	7	1	0	4.7
double	15	7	1	7	10	16.6
double	15	7	1	7	5	10.9
single	15	6	1	6	0	6.0
single	15	6	6	1	0	3.8
double	15	6	1	6	10	14.9
double	15	6	1	6	5	9.7
single	15	5	1	5	0	4.1
single	15	5	5	1	0	2.6
double	15	5	1	5	10	12.9
double	15	5	1	5	4	7.4
single	15	4	1	4	0	2.8
single	15	4	4	1	0	2.1
double	15	4	1	4	10	11.2
double	15	4	1	4	3	5.5
single	15	3	1	3	0	2.1
single	15	3	3	1	0	1.6
double	15	3	1	3	10	9.5
double	15	3	1	3	2	3.9
single	15	2	1	2	0	1.5
single	15	2	2	1	0	1.2
double	15	2	1	2	10	7.8
double	15	2	1	2	2	3.1
single	15	1	1	1	0	1.0
single	15	1	1	1	0	1.0
double	15	1	1	1	10	6.0
double	15	1	1	1	1	1.8
single	16	28	1	28	0	136.0
single	16	28	1	28	0	136.0
double	16	28	1	28	20	143.3
double	16	28	1	28	20	143.3
single	16	27	1	27	0	127.5
single	16	27	2	14	0	94.8
double	16	27	1	27	20	138.8
double	16	27	1	27	19	133.2
single	16	26	1	26	0	118.1
single	16	26	3	9	0	77.4

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	16	26	1	26	20	134.1
double	16	26	1	26	19	128.6
single	16	25	1	25	0	109.1
single	16	25	4	7	0	68.9
double	16	25	1	25	20	129.6
double	16	25	1	25	18	119.2
single	16	24	1	24	0	102.2
single	16	24	5	5	0	60.7
double	16	24	1	24	20	125.1
double	16	24	1	24	17	110.1
single	16	23	1	23	0	94.6
single	16	23	6	4	0	53.1
double	16	23	1	23	20	120.5
double	16	23	1	23	17	106.0
single	16	22	1	22	0	86.5
single	16	22	6	4	0	49.9
double	16	22	1	22	20	116.1
double	16	22	1	22	16	97.6
single	16	21	1	21	0	78.6
single	16	21	7	3	0	43.0
double	16	21	1	21	20	111.2
double	16	21	1	21	15	89.0
single	16	20	1	20	0	71.5
single	16	20	7	3	0	40.2
double	16	20	1	20	20	106.8
double	16	20	1	20	14	81.1
single	16	19	1	19	0	65.4
single	16	19	10	2	0	35.6
double	16	19	1	19	20	102.0
double	16	19	1	19	14	77.5
single	16	18	1	18	0	60.0
single	16	18	9	2	0	31.9
double	16	18	1	18	20	97.6
double	16	18	1	18	13	70.3
single	16	17	1	17	0	54.6
single	16	17	9	2	0	29.7
double	16	17	1	17	20	93.2
double	16	17	1	17	12	63.3
single	16	16	1	16	0	49.6
single	16	16	8	2	0	26.3
double	16	16	1	16	20	88.5
double	16	16	1	16	12	60.1

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	16	15	1	15	0	42.7
single	16	15	8	2	0	24.3
double	16	15	1	15	20	84.0
double	16	15	1	15	11	53.5
single	16	14	1	14	0	38.1
single	16	14	14	1	0	20.0
double	16	14	1	14	20	79.4
double	16	14	1	14	10	47.4
single	16	13	1	13	0	34.0
single	16	13	13	1	0	17.8
double	16	13	1	13	20	75.0
double	16	13	1	13	10	44.7
single	16	12	1	12	0	29.6
single	16	12	12	1	0	15.4
double	16	12	1	12	20	70.1
double	16	12	1	12	9	38.7
single	16	11	1	11	0	24.9
single	16	11	11	1	0	13.4
double	16	11	1	11	20	65.7
double	16	11	1	11	8	33.6
single	16	10	1	10	0	20.4
single	16	10	10	1	0	11.4
double	16	10	1	10	20	60.8
double	16	10	1	10	7	28.4
single	16	9	1	9	0	16.9
single	16	9	9	1	0	9.8
double	16	9	1	9	20	56.4
double	16	9	1	9	7	26.2
single	16	8	1	8	0	14.3
single	16	8	8	1	0	8.2
double	16	8	1	8	20	52.0
double	16	8	1	8	6	21.9
single	16	7	1	7	0	12.0
single	16	7	7	1	0	6.8
double	16	7	1	7	20	47.5
double	16	7	1	7	5	17.9
single	16	6	1	6	0	9.9
single	16	6	6	1	0	5.7
double	16	6	1	6	20	43.0
double	16	6	1	6	5	16.1
single	16	5	1	5	0	6.4
single	16	5	5	1	0	3.9

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	16	5	1	5	20	37.9
double	16	5	1	5	4	12.1
single	16	4	1	4	0	4.5
single	16	4	4	1	0	3.1
double	16	4	1	4	20	33.5
double	16	4	1	4	3	9.1
single	16	3	1	3	0	3.4
single	16	3	3	1	0	2.5
double	16	3	1	3	20	29.1
double	16	3	1	3	2	6.4
single	16	2	1	2	0	2.5
single	16	2	2	1	0	2.0
double	16	2	1	2	20	24.7
double	16	2	1	2	2	5.3
single	16	1	1	1	0	1.7
single	16	1	1	1	0	1.7
double	16	1	1	1	20	20.1
double	16	1	1	1	1	3.1
single	17	57	1	57	0	1081.0
single	17	57	1	57	0	1086.5
double	17	57	1	57	41	890.0
double	17	57	1	57	41	890.1
single	17	56	1	56	0	1051.9
single	17	56	2	28	0	737.1
double	17	56	1	56	41	876.0
double	17	56	1	56	40	857.7
single	17	55	1	55	0	1013.4
single	17	55	3	19	0	618.9
double	17	55	1	55	41	861.5
double	17	55	1	55	39	825.9
single	17	54	1	54	0	981.3
single	17	54	4	14	0	547.7
double	17	54	1	54	41	846.9
double	17	54	1	54	39	814.3
single	17	53	1	53	0	939.5
single	17	53	5	11	0	500.8
double	17	53	1	53	41	831.5
double	17	53	1	53	38	783.3
single	17	52	1	52	0	903.8
single	17	52	6	9	0	464.3
double	17	52	1	52	41	817.7
double	17	52	1	52	37	752.3

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	17	51	1	51	0	870.6
single	17	51	7	8	0	441.8
double	17	51	1	51	41	803.0
double	17	51	1	51	36	720.9
single	17	50	1	50	0	838.1
single	17	50	8	7	0	419.4
double	17	50	1	50	41	788.3
double	17	50	1	50	36	707.6
single	17	49	1	49	0	802.8
single	17	49	9	6	0	394.6
double	17	49	1	49	41	774.9
double	17	49	1	49	35	677.4
single	17	48	1	48	0	771.4
single	17	48	10	5	0	369.0
double	17	48	1	48	41	760.8
double	17	48	1	48	34	647.3
single	17	47	1	47	0	741.7
single	17	47	10	5	0	357.4
double	17	47	1	47	41	746.4
double	17	47	1	47	34	635.3
single	17	46	1	46	0	717.3
single	17	46	12	4	0	334.4
double	17	46	1	46	41	731.2
double	17	46	1	46	33	606.8
single	17	45	1	45	0	686.8
single	17	45	12	4	0	322.8
double	17	45	1	45	41	716.9
double	17	45	1	45	32	579.4
single	17	44	1	44	0	655.7
single	17	44	11	4	0	306.5
double	17	44	1	44	41	702.4
double	17	44	1	44	31	552.6
single	17	43	1	43	0	628.5
single	17	43	15	3	0	289.1
double	17	43	1	43	41	688.2
double	17	43	1	43	31	540.6
single	17	42	1	42	0	593.6
single	17	42	14	3	0	273.7
double	17	42	1	42	41	673.7
double	17	42	1	42	30	514.4
single	17	41	1	41	0	566.1
single	17	41	14	3	0	263.9

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	17	41	1	41	41	658.2
double	17	41	1	41	29	489.3
single	17	40	1	40	0	540.7
single	17	40	14	3	0	254.3
double	17	40	1	40	41	667.7
double	17	40	1	40	29	478.7
single	17	39	1	39	0	516.3
single	17	39	13	3	0	239.3
double	17	39	1	39	41	636.9
double	17	39	1	39	28	454.5
single	17	38	1	38	0	492.5
single	17	38	19	2	0	221.1
double	17	38	1	38	41	628.4
double	17	38	1	38	27	430.7
single	17	37	1	37	0	467.1
single	17	37	19	2	0	213.7
double	17	37	1	37	41	599.5
double	17	37	1	37	27	420.5
single	17	36	1	36	0	443.4
single	17	36	18	2	0	200.6
double	17	36	1	36	41	586.5
double	17	36	1	36	26	398.0
single	17	35	1	35	0	421.9
single	17	35	18	2	0	193.5
double	17	35	1	35	41	571.4
double	17	35	1	35	25	375.5
single	17	34	1	34	0	399.0
single	17	34	17	2	0	180.8
double	17	34	1	34	41	556.5
double	17	34	1	34	24	354.2
single	17	33	1	33	0	372.8
single	17	33	17	2	0	174.1
double	17	33	1	33	41	542.4
double	17	33	1	33	24	345.3
single	17	32	1	32	0	352.4
single	17	32	16	2	0	162.0
double	17	32	1	32	41	527.3
double	17	32	1	32	23	323.9
single	17	31	1	31	0	327.6
single	17	31	16	2	0	155.2
double	17	31	1	31	41	512.6
double	17	31	1	31	22	303.8

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	17	30	1	30	0	307.3
single	17	30	15	2	0	144.6
double	17	30	1	30	41	497.6
double	17	30	1	30	22	295.3
single	17	29	1	29	0	288.1
single	17	29	29	1	0	131.3
double	17	29	1	29	41	483.5
double	17	29	1	29	21	276.1
single	17	28	1	28	0	270.4
single	17	28	28	1	0	123.1
double	17	28	1	28	41	469.0
double	17	28	1	28	20	257.2
single	17	27	1	27	0	254.5
single	17	27	27	1	0	115.0
double	17	27	1	27	41	454.3
double	17	27	1	27	19	239.9
single	17	26	1	26	0	237.9
single	17	26	26	1	0	108.6
double	17	26	1	26	41	439.5
double	17	26	1	26	19	231.9
single	17	25	1	25	0	219.5
single	17	25	25	1	0	101.3
double	17	25	1	25	41	425.0
double	17	25	1	25	18	215.0
single	17	24	1	24	0	203.6
single	17	24	24	1	0	93.7
double	17	24	1	24	41	410.2
double	17	24	1	24	17	198.7
single	17	23	1	23	0	189.0
single	17	23	23	1	0	86.8
double	17	23	1	23	41	396.0
double	17	23	1	23	17	191.4
single	17	22	1	22	0	169.6
single	17	22	22	1	0	80.5
double	17	22	1	22	41	381.7
double	17	22	1	22	16	176.5
single	17	21	1	21	0	154.3
single	17	21	21	1	0	73.4
double	17	21	1	21	41	366.6
double	17	21	1	21	15	161.2
single	17	20	1	20	0	139.9
single	17	20	20	1	0	68.3

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
double	17	20	1	20	41	352.2
double	17	20	1	20	14	147.1
single	17	19	1	19	0	128.1
single	17	19	19	1	0	62.3
double	17	19	1	19	41	337.8
double	17	19	1	19	14	140.7
single	17	18	1	18	0	117.2
single	17	18	18	1	0	56.9
double	17	18	1	18	41	323.6
double	17	18	1	18	13	127.7
single	17	17	1	17	0	107.3
single	17	17	17	1	0	51.6
double	17	17	1	17	41	308.9
double	17	17	1	17	12	115.2
single	17	16	1	16	0	97.3
single	17	16	16	1	0	46.2
double	17	16	1	16	41	294.4
double	17	16	1	16	12	109.8
single	17	15	1	15	0	85.3
single	17	15	15	1	0	42.2
double	17	15	1	15	41	279.6
double	17	15	1	15	11	97.6
single	17	14	1	14	0	75.2
single	17	14	14	1	0	37.6
double	17	14	1	14	41	265.3
double	17	14	1	14	10	86.5
single	17	13	1	13	0	66.4
single	17	13	13	1	0	33.1
double	17	13	1	13	41	250.9
double	17	13	1	13	10	82.0
single	17	12	1	12	0	58.2
single	17	12	12	1	0	28.7
double	17	12	1	12	41	235.8
double	17	12	1	12	9	71.0
single	17	11	1	11	0	47.0
single	17	11	11	1	0	24.9
double	17	11	1	11	41	222.1
double	17	11	1	11	8	61.8
single	17	10	1	10	0	38.2
single	17	10	10	1	0	21.3
double	17	10	1	10	41	206.6
double	17	10	1	10	7	52.4

Type	$\log_2 N$	ℓ	k	ω	$\tilde{\omega}$	Time (ms)
single	17	9	1	9	0	32.0
single	17	9	9	1	0	18.3
double	17	9	1	9	41	192.3
double	17	9	1	9	7	48.6
single	17	8	1	8	0	26.9
single	17	8	8	1	0	15.4
double	17	8	1	8	41	178.1
double	17	8	1	8	6	40.6
single	17	7	1	7	0	22.5
single	17	7	7	1	0	13.0
double	17	7	1	7	41	163.7
double	17	7	1	7	5	33.5
single	17	6	1	6	0	18.7
single	17	6	6	1	0	10.9
double	17	6	1	6	41	149.0
double	17	6	1	6	5	30.2
single	17	5	1	5	0	12.3
single	17	5	5	1	0	7.6
double	17	5	1	5	41	133.7
double	17	5	1	5	4	23.1
single	17	4	1	4	0	8.8
single	17	4	4	1	0	6.1
double	17	4	1	4	41	119.2
double	17	4	1	4	3	17.2
single	17	3	1	3	0	6.5
single	17	3	3	1	0	4.8
double	17	3	1	3	41	105.3
double	17	3	1	3	2	12.2
single	17	2	1	2	0	4.7
single	17	2	2	1	0	3.8
double	17	2	1	2	41	93.0
double	17	2	1	2	2	10.0
single	17	1	1	1	0	3.2
single	17	1	1	1	0	3.2
double	17	1	1	1	41	77.9
double	17	1	1	1	1	6.0

Table 10: M3 Pro execution times for $\omega = 1$ and $\omega = 2$ where q uses approximately half of the available modulus space (see also Table 3).

Parameters		Time (ms)	
$\log_2 N$	$\log_2 q$	$\omega = 1$	$\omega = 2$
14	180 (41.86 %)	1.2	1.4
15	420 (48.39 %)	4.6	5.3
16	840 (48.08 %)	20.1	21.3
17	1740 (49.39 %)	131.2	138.5

Table 11: M3 Pro execution times for parameter sets where $k = 2$ or $k = 1$ with q using the rest of the available modulus space. Each parameter set has a sibling with degree $2N$ and $\omega' = 1$. We also report the speed-up of $2N$ compared to N (see also Table 4).

Parameters				Time (ms)		
$\log_2 N$	$\log_2 q$	ω	ω'	N	$2N$	Speedup
14	300 (69.77 %)	3	1	2.8	2.6	1.05
14	360 (83.72 %)	6	1	4.4	3.8	1.17
15	720 (82.95 %)	6	1	14.3	15.5	0.92
15	780 (89.86 %)	13	1	21.2	17.7	1.20
16	1620 (92.73 %)	14	1	98.3	111.6	0.88
16	1680 (96.16 %)	28	1	148.7	118.1	1.26
17	3360 (95.37 %)	28	1	719.8	788.6	0.91
17	3420 (97.08 %)	57	1	1086.1	817.7	1.33

Table 13: Absolute execution times comparing only small and mostly large primes (see also Table 7).

Parameters				Time (ms)		
$\log_2 N$	ℓ_{36}	ℓ_{54}	ω	`small`	`large`	Speedup
14	9	6	6	6.9	4.4	1.55
14	6	4	2	2.9	1.9	1.56
14	3	2	1	1.0	0.9	1.16
15	21	14	14	42.2	24.1	1.75
15	18	12	4	23.3	13.0	1.80
15	15	10	2	14.6	8.4	1.72
15	12	8	2	10.0	6.1	1.64
15	9	6	1	5.0	3.8	1.33
15	6	4	1	2.9	2.1	1.41

$\log_2 N$	ℓ_{36}	ℓ_{54}	ω	`small`	`large`	Speedup
15	3	2	1	1.4	1.2	1.14
16	45	30	30	275.8	155.1	1.78
16	42	28	10	170.1	89.8	1.89
16	39	26	6	134.6	71.2	1.89
16	36	24	4	106.4	56.7	1.88
16	33	22	3	87.7	48.3	1.82
16	30	20	2	66.1	38.1	1.73
16	27	18	2	55.0	31.8	1.73
16	24	16	2	44.9	26.4	1.70
16	21	14	1	28.5	20.0	1.43
16	18	12	1	22.2	15.4	1.44
16	15	10	1	16.4	11.3	1.45
16	12	8	1	11.5	8.2	1.41
16	9	6	1	7.5	5.7	1.31
16	6	4	1	4.5	3.1	1.44
16	3	2	1	2.2	2.0	1.11
17	93	62	21	1488.6	770.8	1.93
17	90	60	12	1244.7	627.3	1.98
17	87	58	9	1117.5	563.9	1.98
17	84	56	7	999.2	505.1	1.98
17	81	54	5	892.0	454.0	1.96
17	78	52	4	802.4	413.1	1.94
17	75	50	4	750.6	387.0	1.94
17	72	48	3	659.1	346.2	1.90
17	69	46	3	609.3	312.0	1.95
17	66	44	3	562.5	298.8	1.88
17	63	42	2	476.7	265.7	1.79
17	60	40	2	435.2	242.6	1.79
17	57	38	2	394.7	221.2	1.78
17	54	36	2	357.4	200.6	1.78
17	51	34	2	321.5	180.6	1.78
17	48	32	1	231.4	156.3	1.48
17	45	30	1	206.0	139.7	1.48
17	42	28	1	180.8	123.2	1.47
17	39	26	1	158.7	108.5	1.46
17	36	24	1	136.6	94.0	1.45
17	33	22	1	117.9	80.4	1.47
17	30	20	1	98.8	68.0	1.45
17	27	18	1	82.9	56.9	1.46
17	24	16	1	67.7	46.4	1.46
17	21	14	1	53.2	37.7	1.41
17	18	12	1	41.7	28.7	1.45

$\log_2 N$	ℓ_{36}	ℓ_{54}	ω	`small`	`large`	Speedup
17	15	10	1	31.0	21.3	1.46
17	12	8	1	21.5	15.5	1.39
17	9	6	1	14.3	10.9	1.31
17	6	4	1	8.7	6.0	1.46
17	3	2	1	4.2	3.8	1.10

Table 14: Absolute M3 Pro execution times and overhead of switching out large for small primes (see also Table 8).

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`folded`	`switch`	Overhead
14	6	6	4.4	7.9	3.5
14	4	2	1.9	4.6	2.7
14	2	1	0.9	2.7	1.8
15	14	14	24.1	32.6	8.5
15	12	4	13.0	19.6	6.6
15	10	2	8.4	14.2	5.8
15	8	2	6.1	10.8	4.7
15	6	1	3.8	7.5	3.7
15	4	1	2.1	5.1	3.0
15	2	1	1.2	3.2	2.0
16	30	30	155.1	173.3	18.3
16	28	10	89.8	107.5	17.7
16	26	6	71.2	87.4	16.1
16	24	4	56.7	71.9	15.1
16	22	3	48.3	62.3	14.1
16	20	2	38.1	51.0	12.9
16	18	2	31.8	43.8	11.9
16	16	2	26.4	37.0	10.6
16	14	1	20.0	29.7	9.7
16	12	1	15.4	23.9	8.5
16	10	1	11.3	18.7	7.4
16	8	1	8.2	14.6	6.4
16	6	1	5.7	10.7	5.0
16	4	1	3.1	7.5	4.4
16	2	1	2.0	4.8	2.8
17	64	64	1338.2	1427.9	89.6
17	62	21	770.8	840.4	69.6
17	60	12	627.3	690.1	62.8
17	58	9	563.9	623.2	59.3
17	56	7	505.1	560.1	55.0

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`folded`	`switch`	Overhead
17	54	5	454.0	509.3	55.3
17	52	4	413.1	466.5	53.4
17	50	4	387.0	439.8	52.8
17	48	3	346.2	397.8	51.6
17	46	3	312.0	374.3	62.4
17	44	3	298.8	344.5	45.8
17	42	2	265.7	309.5	43.8
17	40	2	242.6	284.6	42.0
17	38	2	221.2	261.2	40.0
17	36	2	200.6	238.7	38.1
17	34	2	180.6	217.2	36.5
17	32	1	156.3	191.5	35.2
17	30	1	139.7	172.2	32.6
17	28	1	123.2	153.9	30.7
17	26	1	108.5	136.1	27.6
17	24	1	94.0	120.6	26.6
17	22	1	80.4	105.5	25.1
17	20	1	68.0	90.6	22.6
17	18	1	56.9	78.0	21.1
17	16	1	46.4	65.9	19.5
17	14	1	37.7	55.4	17.6
17	12	1	28.7	44.8	16.1
17	10	1	21.3	35.0	13.7
17	8	1	15.5	27.7	12.2
17	6	1	10.9	20.2	9.3
17	4	1	6.0	14.1	8.1
17	2	1	3.8	9.1	5.3

Table 15: Speedups of constant foldings for different ω (see also Table 5).

$\log_2 N$	ω				
	1	2	3	4	5
14	1.25×	1.17×	1.21×	1.15×	1.11×
15	1.13×	1.11×	1.09×	1.11×	1.08×
16	1.07×	1.03×	1.03×	1.03×	1.03×
17	1.03×	1.03×	1.02×	1.02×	1.02×

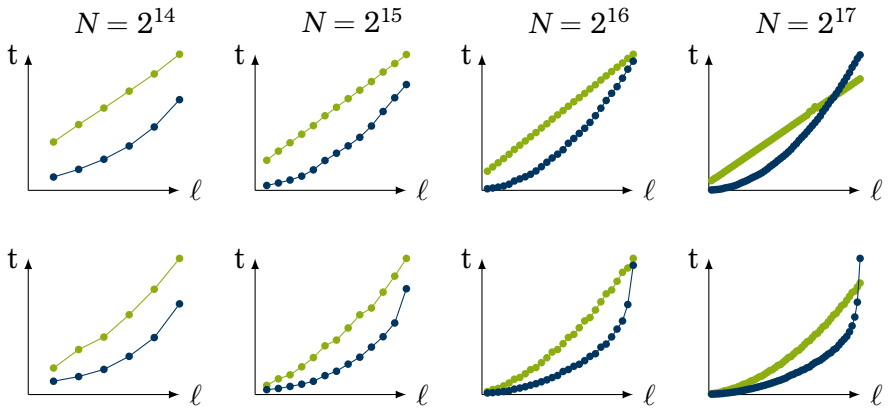

Figure 2: Normalized M3 Pro execution times per number of primes ℓ for the ● single-decomposition and ● double-decomposition techniques (lower is better). We either fix $k = 1, \omega = L$, and $\tilde{\omega} = \sqrt{L(L+1)/2}$ maximizing q (top) or set these optimal per ℓ (bot). See also Figure 1. The absolute timings are listed in Table 12.

Table 16: Absolute M3 Pro execution times of folding speedups (see also Table 9).

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`naïve`	`folded`	Speedup
14	3	1	1.5	1.2	1.25
14	4	2	2.2	1.9	1.17
14	3	3	1.8	1.5	1.21
14	4	4	2.5	2.2	1.15
14	5	5	3.4	3.1	1.11
14	6	6	4.8	4.4	1.09
15	7	1	5.2	4.7	1.13
15	8	2	6.7	6.1	1.11
15	9	3	8.4	7.7	1.09
15	8	4	7.7	7.0	1.11
15	10	5	10.9	10.1	1.08
15	12	6	15.3	14.3	1.07
15	7	7	8.1	7.4	1.09
15	8	8	9.8	8.9	1.10
15	9	9	11.5	10.6	1.08
15	10	10	13.8	12.9	1.07
16	14	1	21.5	20.1	1.07
16	18	2	33.0	31.9	1.03
16	21	3	44.4	42.9	1.03
16	20	4	43.1	41.7	1.03
16	20	5	45.0	43.7	1.03

Parameters			Time (ms)		
$\log_2 N$	ℓ	ω	`naïve`	`folded`	Speedup
16	24	6	63.0	61.0	1.03
16	21	7	52.6	51.1	1.03
16	24	8	67.4	65.6	1.03
16	18	9	45.1	43.9	1.03
16	20	10	54.5	53.2	1.02
17	29	1	135.5	131.3	1.03
17	38	2	226.9	221.1	1.03
17	42	3	279.1	273.9	1.02
17	44	4	312.8	306.3	1.02
17	45	5	334.1	328.0	1.02
17	48	6	383.1	376.5	1.02
17	49	7	406.4	398.8	1.02
17	48	8	402.0	393.9	1.02
17	45	9	367.5	361.4	1.02
17	50	10	450.3	442.0	1.02